

# claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_2ae09dfb-e3e\agent-a45b94005a985e754.jsonl`
- SHA-256: `5d8c4ab61968d6005720c1f78ad70babce3c3548d5101d13bf668b83cf15dd18`
- Source modified: `2026-06-13T20:04:38+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_2ae09dfb-e3e`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

## Transcript

### 1. user (2026-06-13T20:00:01.296Z)

Synthesize a CONCRETE, buildable plan for fusion P3 (learned fusion + LOVO lift measurement on the 6 golden videos) and P4 (audio cue) from these maps (JSON): [{"findings":"### P3 FUSION LEARNED-LOVO SURFACE MAP: Shuttle-Only vs Shuttle+Person F1 on 6 Golden Videos\n\n### FEATURE LANDSCAPE\n\*\*Shuttle features (5)\*\* ? already in `distill\_local.FEATURE\_NAMES` (line 40):\n- `duration`: rally length (sec)\n- `peak\_velocity`: max per-frame shuttle speed, frame-width-normalised\n- `mean\_velocity`: avg shuttle speed\n- `active\_ratio`: active\_frame\_count / window\_frames\n- `net\_crossings`: net-crossing count (Gate A signal, from `rally\_gate.count\_net\_crossings`)\n\n\*\*Person features (5)\*\* ? defined in `fusion\_features.PERSON\_FEATURE\_NAMES` (lines 194?197):\n- `players\_in\_court`: median per-frame in-court count (scale-invariant, 0?2+ players)\n- `two\_sided\_motion`: fraction of frames with >=1 person on BOTH net halves (detects one-sided feeds)\n- `mean\_player\_motion`: mean per-track centre displacement, normalised by frame dims\n- `in\_court\_ratio`: fraction of sampled frames with >=2 in-court players\n- `shuttle\_player\_colocation`: fraction of visible shuttle frames inside/adjacent to a person box (held-shuttle signal)\n\n### DATA FLOW: HOW TO BUILD LABELED FUSION CANDIDATES\n\n\*\*Step 1: Load persisted candidates from DB\*\*\n\n```\nexperiment.load\_video\_candidates(db, video\_id, \n detector=\"wasb\", # shuttle substrate\n feature\_names=[\"duration\", \"peak\_velocity\", \"mean\_velocity\", \"active\_ratio\", \"net\_crossings\", \"players\_in\_court\", \"two\_sided\_motion\", \"mean\_player\_motion\", \"in\_court\_ratio\", \"shuttle\_player\_colocation\"],\n gts=golden\_rally\_intervals)\nVideoCandidates(name, intervals, features: {feat\_name ? [values...]}, gts)\n```\n\n\*\*Reference\*\*:\n`experiment.py:429?447` (`load\_video\_candidates`)\n\nPersisted features come from:\n- \*\*Shuttle features\*\*: `FusionSegmenter.\_enrich\_candidate\_stats()` (fusion\_hybrid.py:76?90) calls:\n- `TrackNetRunner.trajectory\_to\_action\_windows()` ? duration, peak/mean velocity, active\_ratio (from WASB)\n- `rally\_gate.gate\_windows()` ? net\_crossings (line 85)\n\n- \*\*Person features\*\*: Only if `indexing.fusion.cue\_flags.person=True` ?\n`FusionSegmenter.\_person\_features()` (fusion\_hybrid.py:103?131):\n- Calls `PersonDetector.detections\_per\_frame(video\_path, start\_f, end\_f, frame\_skip)` (person\_detector.py:258?303)\n- Returns `{\"frames\": {frame\_idx: [(track\_id, bbox), ...], ...}}`\n- Passed to `fusion\_features.compute\_person\_features()` (fusion\_features.py:200?215) ? Dict[str, float] with all 5 person features\n\n\*\*Step 2: Label candidates by IoU matching\*\*\n\n```\ntraining.label\_candidates(candidate\_intervals, golden\_gts, iou\_threshold=0.5)\n`[1 if matched to GT else 0 for each candidate]`\n```\n\n\*\*Reference\*\*:\n`training.py:50?61` ? uses the same `metrics.match\_intervals()` as the evaluator, so training labels are consistent with scoring.\n\n\*\*Result\*\*: VideoCandidates with:\n- `intervals`: candidate windows\n- `features`: 10 feature columns (5 shuttle + 5 person)\n- `gts`: golden labels\n- Per-candidate binary label `y[i]` derives from `label\_candidates(intervals, gts, 0.5)`\n\n### MEASURING HELD-OUT LIFT: LOVO WITH BOTH FEATURE SETS\n\n\*\*Strategy\*\*: Honest leave-one-video-out, train two variants (shuttle-only vs fusion), report per-video + aggregate F1 delta.\n\n### Option 1: EXTEND `distill\_local.run()` (Minimal code change)\n\n\*\*File\*\*:\n`backend/eval/distill\_local.py:98?143`\n\nAdd a function:\n\n```python\ndef run\_fusion\_comparison(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5) -> dict:\n \"\"\"LOVO shuttle-only vs shuttle+person fusion on the 6 golden videos.\"\"\"\n from backend.eval.experiment import VideoCandidates, Variant, evaluate\_variant\n # Build

```
VideoCandidates with BOTH feature sets from the DB or pre-loaded candidates\nshuttle_videos = [...] # load with ["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n fusion_videos = [...] # load with all 10 features\n\n # Variant A: shuttle-only learned filter\n shuttle_variant = Variant(\n     name="shuttle_only",\n     feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"],\n     threshold=0.5\n )\n\n # Variant B: shuttle+person fusion\n fusion_variant = Variant(\n     name="shuttle_person_fusion",\n     feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                    "players_in_court", "two_sided_motion", "mean_player_motion",\n                    "in_court_ratio",\n                    "shuttle_player_colocation"],\n     threshold=0.5\n )\n\n # Honest LOVO on both\nshuttle_result = evaluate_variant(shuttle_videos, shuttle_variant, iou=iou, honest=True)\n fusion_result = evaluate_variant(fusion_videos, fusion_variant, iou=iou, honest=True)\n\n # Report per-video + aggregate F1 deltas\n print(f"Shuttle-only: F1={shuttle_result['aggregate']['f1']:.3f}")\n print(f"Fusion: F1={fusion_result['aggregate']['f1']:.3f}")\n for sv, fv in zip(shuttle_result['per_video'], fusion_result['per_video']):\n     print(f" {sv['video']}: ?F1={fv['f1'] - sv['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- Reuses the existing `training.label_candidates` (line 84) and `LogisticRegression.fit()` harness.\n- `evaluate_variant()` (experiment.py:245?300) already implements honest LOVO per-fold training.\n- Clear, unit-tested, low risk.\n\n**Seam**: Call this from a new CLI entry point (e.g., `python -m backend.eval.fusion_compare --manifest videos.json`).\n\n---\n\n#### Option 2: USE `experiment.compare()` DIRECTLY (Recommended for one-off report)\n\n**File**:\n`backend/eval/experiment.py:303?339`\n\n```\npython\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, CONTROL\n\n# Load 6 golden videos with all 10 features (assuming FusionSegmenter ran with person=True)\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"\n    ], gts=golden_gts[vid])\n]\n\nfor vid in ["video_id_1", ..., "video_id_6"]:\n    shuttle_variant = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion_variant = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\n    result = compare(fusion_videos, shuttle_variant, fusion_variant, iou=0.5, honest=True)\n    # result["delta"]["f1"] = fusion F1 - shuttle F1 (held-out honest LOVO)\n    print(f"Held-out LOVO F1 lift: {result['delta']['f1']:+.3f}")\n\nfor row in result["per_video_delta"]:\n    print(f" {row['video']}: {row['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- `compare()` (experiment.py:303?339) does:\n    - Train both variants per LOVO fold on the other N-1 videos (lines 314?315).\n    - Predict and score the held-out video (lines 314?315).\n    - Report per-video F1 deltas (lines 320?324) + aggregate pct improvement (line 318).\n    - Label-set fingerprint (line 332) ? detects if you measured against different labels.\n- Already shipped, tested (`test_experiment.py:149?156`).\n- Produces a leaderboard-ready record (`record_experiment()`, line 395).\n\n**Seam**: A standalone script, e.g., `backend/eval/fusion_compare.py`.\n\n---\n\n#### Option 3: BRIDGE VIA A NEW EVAL DRIVER (Cleanest integration)\n\n**File**: New file `backend/eval/fusion_lovo_driver.py` (30 lines, pure)\n\n```\npython\nfrom typing import List, Dict, Any\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, record_experiment, DEFAULT_LEADERBOARD_PATH\n\n# Measure shuttle-only vs shuttle+person fusion LOVO F1 on labeled videos.\nimport json\n\n# Load 6 golden videos with all 10 features\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"]\n    ], gts=golden_gts.get(vid))\n]\n\nfor vid in video_ids:\n    shuttle_only = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\n    result = compare(fusion_videos, shuttle_only, fusion, iou=0.5, honest=True)\n    # result["delta"]["f1"] = fusion F1 - shuttle F1 (held-out honest LOVO)\n    print(f"Held-out LOVO F1 lift: {result['delta']['f1']:+.3f}")\n\nfor row in result["per_video_delta"]:\n    print(f" {row['video']}: {row['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- `compare()` (experiment.py:303?339) does:\n    - Train both variants per LOVO fold on the other N-1 videos (lines 314?315).\n    - Predict and score the held-out video (lines 314?315).\n    - Report per-video F1 deltas (lines 320?324) + aggregate pct improvement (line 318).\n    - Label-set fingerprint (line 332) ? detects if you measured against different labels.\n- Already shipped, tested (`test_experiment.py:149?156`).\n- Produces a leaderboard-ready record (`record_experiment()`, line 395).\n\n**Seam**: A standalone script, e.g., `backend/eval/fusion_compare.py`.\n\n---\n\n#### Option 3: BRIDGE VIA A NEW EVAL DRIVER (Cleanest integration)\n\n**File**: New file `backend/eval/fusion_lovo_driver.py` (30 lines, pure)\n\n```\npython\nfrom typing import List, Dict, Any\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, record_experiment, DEFAULT_LEADERBOARD_PATH\n\n# Measure shuttle-only vs shuttle+person fusion LOVO F1 on labeled videos.\nimport json\n\n# Load 6 golden videos with all 10 features\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"]\n    ], gts=golden_gts.get(vid))\n]\n\nfor vid in video_ids:\n    shuttle_only = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\n    result = compare(fusion_videos, shuttle_only, fusion, iou=0.5, honest=True)\n    # result["delta"]["f1"] = fusion F1 - shuttle F1 (held-out honest LOVO)\n    print(f"Held-out LOVO F1 lift: {result['delta']['f1']:+.3f}")\n\nfor row in result["per_video_delta"]:\n    print(f" {row['video']}: {row['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- `compare()` (experiment.py:303?339) does:\n    - Train both variants per LOVO fold on the other N-1 videos (lines 314?315).\n    - Predict and score the held-out video (lines 314?315).\n    - Report per-video F1 deltas (lines 320?324) + aggregate pct improvement (line 318).\n    - Label-set fingerprint (line 332) ? detects if you measured against different labels.\n- Already shipped, tested (`test_experiment.py:149?156`).\n- Produces a leaderboard-ready record (`record_experiment()`, line 395).\n\n**Seam**: A standalone script, e.g., `backend/eval/fusion_compare.py`.\n\n---\n\n#### Option 3: BRIDGE VIA A NEW EVAL DRIVER (Cleanest integration)\n\n**File**: New file `backend/eval/fusion_lovo_driver.py` (30 lines, pure)\n\n```\npython\nfrom typing import List, Dict, Any\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, record_experiment, DEFAULT_LEADERBOARD_PATH\n\n# Measure shuttle-only vs shuttle+person fusion LOVO F1 on labeled videos.\nimport json\n\n# Load 6 golden videos with all 10 features\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"]\n    ], gts=golden_gts.get(vid))\n]\n\nfor vid in video_ids:\n    shuttle_only = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\n    result = compare(fusion_videos, shuttle_only, fusion, iou=0.5, honest=True)\n    # result["delta"]["f1"] = fusion F1 - shuttle F1 (held-out honest LOVO)\n    print(f"Held-out LOVO F1 lift: {result['delta']['f1']:+.3f}")\n\nfor row in result["per_video_delta"]:\n    print(f" {row['video']}: {row['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- `compare()` (experiment.py:303?339) does:\n    - Train both variants per LOVO fold on the other N-1 videos (lines 314?315).\n    - Predict and score the held-out video (lines 314?315).\n    - Report per-video F1 deltas (lines 320?324) + aggregate pct improvement (line 318).\n    - Label-set fingerprint (line 332) ? detects if you measured against different labels.\n- Already shipped, tested (`test_experiment.py:149?156`).\n- Produces a leaderboard-ready record (`record_experiment()`, line 395).\n\n**Seam**: A standalone script, e.g., `backend/eval/fusion_compare.py`.\n\n---\n\n#### Option 3: BRIDGE VIA A NEW EVAL DRIVER (Cleanest integration)\n\n**File**: New file `backend/eval/fusion_lovo_driver.py` (30 lines, pure)\n\n```\npython\nfrom typing import List, Dict, Any\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, record_experiment, DEFAULT_LEADERBOARD_PATH\n\n# Measure shuttle-only vs shuttle+person fusion LOVO F1 on labeled videos.\nimport json\n\n# Load 6 golden videos with all 10 features\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"]\n    ], gts=golden_gts.get(vid))\n]\n\nfor vid in video_ids:\n    shuttle_only = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\n    result = compare(fusion_videos, shuttle_only, fusion, iou=0.5, honest=True)\n    # result["delta"]["f1"] = fusion F1 - shuttle F1 (held-out honest LOVO)\n    print(f"Held-out LOVO F1 lift: {result['delta']['f1']:+.3f}")\n\nfor row in result["per_video_delta"]:\n    print(f" {row['video']}: {row['f1']:+.3f}")\n\n```\n\n**Advantages**:\n- `compare()` (experiment.py:303?339) does:\n    - Train both variants per LOVO fold on the other N-1 videos (lines 314?315).\n    - Predict and score the held-out video (lines 314?315).\n    - Report per-video F1 deltas (lines 320?324) + aggregate pct improvement (line 318).\n    - Label-set fingerprint (line 332) ? detects if you measured against different labels.\n- Already shipped, tested (`test_experiment.py:149?156`).\n- Produces a leaderboard-ready record (`record_experiment()`, line 395).\n\n**Seam**: A standalone script, e.g., `backend/eval/fusion_compare.py`.\n\n---\n\n#### Option 3: BRIDGE VIA A NEW EVAL DRIVER (Cleanest integration)\n\n**File**: New file `backend/eval/fusion_lovo_driver.py` (30 lines, pure)\n\n```\npython\nfrom typing import List, Dict, Any\nfrom backend.eval.experiment import compare, load_video_candidates, Variant, record_experiment, DEFAULT_LEADERBOARD_PATH\n\n# Measure shuttle-only vs shuttle+person fusion LOVO F1 on labeled videos.\nimport json\n\n# Load 6 golden videos with all 10 features\nfusion_videos = [\n    load_video_candidates(db, vid, feature_names=[\n        "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n        "players_in_court", "two_sided_motion", "mean_player_motion",\n        "in_court_ratio", "shuttle_player_colocation"]\n    ], gts=golden_gts.get(vid))\n]\n\nfor vid in video_ids:\n    shuttle_only = Variant(\n        name="shuttle_only",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]\n    )\n    fusion = Variant(\n        name="shuttle_person_fusion",\n        feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",\n                        "players_in_court", "two_sided_motion", "mean_player_motion",\n                        "in_court_ratio", "shuttle_player_colocation"]\n    )\
```

```

\"players_in_court\", \"two_sided_motion\", \"mean_player_motion\", \n
\"in_court_ratio\", \"shuttle_player_colocation\"]\n    )\n    \n    return
compare(fusion_videos, shuttle_only, fusion, iou=iou, honest=True)\n\n# Usage:\n# result =
measure_fusion_lift(db, [\"golden_1\", ..., \"golden_6\"], gts_dict)\n# print(f\"Fusion F1 lift:
{result['delta']['f1']:.3f} ({result['delta_pct']['f1']:.1f}%)\")\n\n# record_experiment(result)
# optional: append to leaderboard\n``\n\n**Advantages**:\n- **Reuses zero new machinery**:\n`compare()` + `load_video_candidates()` already do the heavy lifting.\n- **Pluggable**:\nAccepts
any `db` with `query_candidate_segments()` interface.\n- **Leaderboard-ready**:\nResult struct is
compatible with `record_experiment()` (experiment.py:395).\n- **Honest LOVO**:\nVariant recipe
(no pinned model) triggers per-fold training (experiment.py:267?279).\n\n**Seam**:\nCleanest
integration ? calls the shipped `experiment` harness, no new scoring/labeling
logic.\n\n---\n\n### RECIPE: HOW TO INVOKE ON 6 GOLDEN VIDEOS\n\n**Preconditions**:\n1. **Master
has FusionSegmenter + person detector**:\nVerify `backend/pipeline/segmenters/fusion_hybrid.py`
and `backend/pipeline/detectors/person_detector.py` are present (? on master).\n2. **Person cue
enabled**:\nIndexing config must have `indexing.fusion.cue_flags.person: true` (or loader
supplies person features).\n3. **6 golden videos already persisted**:\nRun FusionSegmenter with
person-ON on each of the 6 golden videos to populate `candidate_segments` with the 5 person
features in `stats` JSON.\n\n**Execution** (using Option 2, `experiment.compare()`\n
directly):\n\n``python\n# 1. Load the DB\nfrom backend.infrastructure.database import
Database\nndb = Database(config_path=\"..\")\n\n# 2. Define the 6 golden video IDs and their GT
intervals\ngolden_ids = [\"video_golden_1\", ..., \"video_golden_6\"]\ngolden_gts = {\n    vid:
[interval1, interval2, ...] # [(start, end), ...] from Gemini labels\n    for vid in
golden_ids\n}\n\n# 3. Load VideoCandidates with all 10 features\nfrom backend.eval.experiment
import load_video_candidates\nfusion_videos = [\n    load_video_candidates(db, vid,\n    detector=\"wasb\", # or \"tracknet\", whatever seeded the candidates\n    feature_names=[\n        \"duration\", \"peak_velocity\", \"mean_velocity\", \"active_ratio\", \"net_crossings\", \n        \"players_in_court\", \"two_sided_motion\", \"mean_player_motion\", \n        \"in_court_ratio\", \"shuttle_player_colocation\"]\n    ],\n    gts=golden_gts[vid])\n    for vid in golden_ids\n]\n\n# 4. Define and compare variants\nfrom backend.eval.experiment
import compare, Variant\nshuttle_variant = Variant(\n    name=\"shuttle_only\", \n    feature_names=[\"duration\", \"peak_velocity\", \"mean_velocity\", \"active_ratio\", \n    \"net_crossings\"]\n)\n\nfusion_variant = Variant(\n    name=\"shuttle_person_fusion\", \n    feature_names=[\"duration\", \"peak_velocity\", \"mean_velocity\", \"active_ratio\", \n    \"net_crossings\", \n        \"players_in_court\", \"two_sided_motion\", \n        \"mean_player_motion\", \n        \"in_court_ratio\", \n        \"shuttle_player_colocation\"]\n)\n\nresult = compare(fusion_videos, shuttle_variant,
fusion_variant, iou=0.5, honest=True)\n\n# 5. Report\nprint(\"=== Honest Leave-One-Video-Out
===\")\nprint(f\"Shuttle-only:
F1={result['variant_a']['aggregate']['f1']:.3f}\")\nprint(f\"Fusion (A+B):
F1={result['variant_b']['aggregate']['f1']:.3f}\")\nprint(f\"LOVO lift:
?F1={result['delta']['f1']:.3f} ({result['delta_pct']['f1']:.1f}%)\")\nprint(\"\\n\\nPer-video
deltas:\")\nfor row in result[\"per_video_delta\"]:\n    print(f\"    {row['video']}:
?F1={row['f1']:.3f}, ?P={row['precision']:.3f}, ?R={row['recall']:.3f}\")\n\n# 6. Optionally
record to leaderboard\nfrom backend.eval.experiment import
record_experiment\nrecord_experiment(result)\n``\n\n---\n\n### THE LOVO LOOP (WHAT HAPPENS
INSIDE `experiment.compare()`)\n\n**File reference**:\n`experiment.py:245?300`
(`evaluate_variant`) + `experiment.py:303?339` (`compare`)\n\nFor each of the 6 videos (held-out
= video i):\n1. **Train phase** (experiment.py:275?279):\n    - Pool candidates from the OTHER 5
videos.\n    - Call `training.label_candidates(intervals, gts, 0.5)` on each
(training.py:50?61).\n    - Stack X (feature matrix), y (binary labels).\n    - Fit
`LogisticRegression` per variant:\n        - **Variant A (shuttle-only)**: 5 features\n        -
**Variant B (fusion)**: 10 features\n    - Both trained on the same 5-video pool (honest).\n2.
**Predict phase** (experiment.py:284):\n    - Call `predict(variant, held_out_video,
model=trained_model)` (experiment.py:203?224).\n    - For the held-out video i, extract its
feature matrix X_held from VideoCandidates (experiment.py:185?187).\n    -
`model.predict_proba(X_held)` ? probabilities.\n    - Apply threshold (0.5) + Gate A/B masks +
merge.\n    - Return predicted rally intervals.\n3. **Score phase** (experiment.py:286):\n    -
Call `metrics.score(preds, held_out_video.gts, iou=0.5)` (metrics.py ~line 100+).\n    - Greedy
matching (metrics.py:48?81), precision/recall/F1 computed per-video.\n4. **Aggregate**
(experiment.py:290?291):\n    - Mean F1 over the 6 held-out scores (one F1 per fold).\n    - Delta
= fusion_f1 ? shuttle_f1 (per-video and aggregate, experiment.py:317?324).\n\n**No new scoring
math**:\nEverything defers to `metrics.score()` (one-to-one IoU matching, standard
precision/recall/F1).\n\n---\n\n### CLEANEST SEAM CHOICE\n\n**Recommended**:\n**Option 2/3
hybrid** ? a ~50-line script that calls `experiment.compare()` with the two Variant
recipes:\n\n``\nbackend/eval/fusion_compare.py [--db-config CONFIG] [--golden-ids ID1,ID2,...]

```

```

[--record]\n` ``\n\n**Why**:\n- **Reuses shipped machinery**:\n`experiment.compare()`,
`load_video_candidates()`, `evaluate_variant()`, `LogisticRegression.fit()` all exist and are
tested.\n- **No new labeling logic**:\n`training.label_candidates()` is the source of truth; both
variants use it.\n- **Honest LOVO guaranteed**:\nVariant recipe (no pinned model) forces per-fold
training (experiment.py:267).\n- **Leaderboard-ready**:\nResult record format matches
`record_experiment()`. \n- **Test surface tiny**:\nJust orchestration; all math is in `experiment`
(already tested extensively, `test_experiment.py`). \n- **Clear failure modes**:\nIf person
features are missing/wrong, they'll appear as 0.0 in the feature matrix (experiment.py:176?182),
and the learned filter will degrade to shuttle-only automatically. \n\n**No new seams needed in
the harness itself** ? the `Variant.feature_names` field (experiment.py:111) was designed for
exactly this: plug in a recipe with different features, `compare()` trains both per-fold,
reports the delta. \n\n---\n\n### EVIDENCE ON MASTER\n\n1. **FusionSegmenter** persists
net_crossings + person features (fusion_hybrid.py:85?89): ? lines 85, 89\n2.
**PersonDetector.detections_per_frame()** returns per-frame detections
(person_detector.py:258?303): ? shipped\n3. **fusion_features.compute_person_features()**
produces the 5 person features (fusion_features.py:200?215): ? lines 194?197
(PERSON_FEATURE_NAMES)\n4. **load_video_candidates()** lifts persisted features into
VideoCandidates (experiment.py:429?447): ? lines 441?446 handle arbitrary feature_names\n5.
**evaluate_variant()** does honest LOVO on learned variants (experiment.py:245?300): ? lines
267?279\n6. **compare()** reports per-video + aggregate deltas (experiment.py:303?339): ? lines
317?324\n7. **Variant.feature_names** is the pluggable seam (experiment.py:111): ? design
already in place\n\nAll building blocks are on master. No new evaluation machinery needed. \n", "a
nchors":["backend/eval/distill_local.py:40", "backend/eval/experiment.py:429-
447", "backend/pipeline/detectors/fusion_features.py:194-
197", "backend/pipeline/segmenters/fusion_hybrid.py:76-
90", "backend/pipeline/detectors/person_detector.py:258-303", "backend/eval/training.py:50-
61", "backend/eval/experiment.py:111", "backend/eval/experiment.py:203-
224", "backend/eval/experiment.py:245-300", "backend/eval/experiment.py:303-
339", "backend/eval/classifier.py:40-73"], "risks":["Person features require PersonDetector model
load (GPU) ? only if person cue is enabled in config; default (person OFF) avoids detector
entirely.", "Feature names must match persisted column names exactly; typos will cause graceful
degradation to 0.0 (experiment.py:176?182) rather than error.", "VideoCandidates.features is
column-major (feature_name ? list); misalignment with intervals length raises ExperimentError
(experiment.py:180?181).", "Honest LOVO requires >=2 videos; single-video test will fail
(experiment.py:278)."]}, {"findings": "### Offline Feature Extraction Pipeline for Golden Videos
(P3 GPU Step)\n\n**Manifest Extension:** Add `video_path` field to
`output/human_lovo_manifest.json` per entry, mapping 6 golden videos:\n- `adarsh_avi_singles` ?
`C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh and Avi.MP4`\n-
`kushagra_singles` ? `KushagraYashAviFirstVideo.MP4`\n- `largetest_doubles` ?
`LargeTestVideo.MP4`\n- `gbaaddy` ? `gBaaddy20May2026.MP4`\n- `testlarge_short` ?
`TestLargeVideo.MP4`\n- `mahadevpura_singles` ? `MahadevpuraSingles.MP4`\n\n**Per-Video Driver
(6 steps):**\n1. Parse manifest + trajectory CSV; build recall-oriented shuttle candidates via
`TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, velocity_thresh=0.005,
merge_gap=1.0, min_window_duration=0.5)` (distill_local.PROPOSAL)\n2. Enrich with shuttle
features: `duration`, `peak_velocity`, `mean_velocity`, `active_frame_count`, `net_crossings`
(via `rally_gate.gate_windows()`)\n3. Per window, run
`PersonDetector.detections_per_frame(video_path, start_frame, end_frame, frame_skip=3)` over
original video frames to get frame-aligned detections\n4. Compute person features via
`fusion_features.compute_person_features(per_frame, shuttle_points, frame_width, frame_height,
court_polygon=None, net=None)` ? `[\"players_in_court\", \"two_sided_motion\",
\"mean_player_motion\", \"in_court_ratio\", \"shuttle_player_colocation\"]`\n5. Label y-values
via `training.label_candidates(intervals, gts, iou=0.5)` from golden CSV (start_time,
end_time)\n6. Persist `{name}_golden_features.json` with candidates array: `[{start_frame,
end_frame, start_time, end_time, shuttle_features, person_features, label,
label_source}]`\n\n**Output Schema:** Per-video `output/{name}_golden_features.json` contains
labeled dataset ready for `experiment.py` harness via `VideoCandidates(name, intervals,
features={feature_name: [per-candidate values]}, gts=[...])`\n\n**Frame Alignment Critical:**
`PersonDetector.detections_per_frame()` reads by absolute frame index (seek + sequential, no re-
extraction); returned frames align with trajectory CSV's frame numbering. Enables
`fusion_features.shuttle_player_colocation()` and `two_sided_motion()`. \n\n**Experiment Harness
Ready:** Persisted features feed `experiment.py:Variant` with `feature_names=[shuttle_features +
person_features]` for LOVO training via `evaluate_variant(videos, variant,
honest=True)`.\n\n**Name-to-File Mapping Table:**\n| Manifest name | Video File | FPS | Width |
Labels CSV |\n|---|---|---|---|---|\n| adarsh_avi_singles | Adarsh and Avi.MP4 | 59.94 | 1920 |
Adarsh and Avi.rallies.csv |\n| kushagra_singles | KushagraYashAviFirstVideo.MP4 | 59.94 | 1920

```

```

| KushagraYashAviFirstVideo.rallies.csv |\n| targetest_doubles | LargeTestVideo.MP4 | 29.97 |
1920 | LargeTestVideo.rallies.csv |\n| gbaaddy | gBaaddy20May2026.MP4 | 59.94 | 1920 |
gBaaddy20May2026.rallies.csv |\n| testlarge_short | TestLargeVideo.MP4 | 29.97 | 1920 |
TestLargeVideo.rallies.csv |\n| mahadevpura_singles | MahadevpuraSingles.MP4 | 29.97 | 1920 |
MahadevpuraSingles.rallies.csv
|", "anchors": ["backend/pipeline/detectors/tracknet_runner.py:252-319
(trjectory_to_action_windows)", "backend/pipeline/detectors/tracknet_runner.py:233-249
(parse_trajectory_csv)", "backend/eval/distill_local.py:40-68 (FEATURE_NAMES,
build_candidates)", "backend/pipeline/detectors/rally_gate.py:91-110 (gate_windows for
net_crossings)", "backend/pipeline/detectors/person_detector.py:258-303 (detections_per_frame
signature: returns {frames: {frame_idx: [(track_id, bbox)]}, fps, frame_width,
frame_height})", "backend/pipeline/detectors/person_detector.py:100-138 (detect_and_track with
ByteTrack)", "backend/pipeline/detectors/fusion_features.py:200-215 (compute_person_features
signature)", "backend/pipeline/detectors/fusion_features.py:194-197 (PERSON_FEATURE_NAMES:
[players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation])", "backend/eval/training.py:50-61 (label_candidates via IoU
matcher)", "backend/eval/experiment.py:154-168 (VideoCandidates
dataclass)", "backend/eval/experiment.py:73-143 (Variant with feature_names, threshold,
merge_gap)", "backend/eval/experiment.py:203-224 (predict function: gate + learned filter +
merge)", "backend/eval/experiment.py:245-300 (evaluate_variant with honest
LOVO)", "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/human_lovo_manifest.json", "C:/Users/avidu/Projects/Annotation
Setup/Collect/Golden Labelled/ (6 MP4 video files)", "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/ (6 trajectory CSV files)"]}, {"findings": "## P4 Audio Hit Detector as
Cue/Feature ? Design Specification\n\n**Status:** Design for integration into FusionSegmenter
(P2) as an enabled-cue branch, default-OFF. \n\n**E1 Result (2026-06-07):** Audio classical onset
detector found ~2.2x discrimination ceiling (in-rally vs dead-time hit density); cannot
distinguish shuttle impact from toss/footstep/voice. **Decision: persist weak signal, let
learned scorer decide.**\n\n--\n\n#### 1. Architecture & Integration Points\n\n##### 1.1 Audio
Hit Detection Producer\n\n**File:** `backend/pipeline/audio/hit_detector.py` (EXISTING ? already
on master) \n\n**Exports:**\n- `detect_hits_in_video(video_path, sr=16000, **kw)` ->
List[HitEvent]` ? line 136\n- Stages 0?1 of AUDIO_RALLY_DETECTION_PLAN (extract audio via
ffmpeg ? onset envelope ? adaptive peak threshold)\n- Returns `(timestamp, strength)` impact
candidates\n- Key params: `k` (threshold sensitivity, default 2.5), `band` (tuple Hz for HF
band-limiting)\n- `HitEvent` dataclass ? line 23: `t: float` (seconds), `strength: float`
(onset-envelope peak)\n\n**Dependencies:** numpy, ffmpeg (subprocess), stdlib `wave` ? NO scipy,
NO heavy models \n\n**CPU cost:** ~0.02x the video duration (ffmpeg audio decode +
STFT)\n\n--\n\n##### 1.2 Audio Features Producer (NEW MODULE)\n\n**Location:**
`backend/pipeline/detectors/audio_features.py` (new) \n\n**Scope:** Transform per-window hit
events ? scale/translation-invariant features, mirroring `fusion_features.py`
discipline\n\n**Module Structure:**\n- `python`"\n- "Audio cue features for fusion rally
detection (P4, AUDIO_RALLY_DETECTION_PLAN $4/$5).\n- Window-level aggregators over hits detected
in the window's timespan. Pure functions\n- (no I/O, no GPU) so features are unit-testable
standalone and featurizable before\n- the classifier runs. Every feature defaults to 0.0 on empty
input (graceful degradation).\n- "\n- "\n- from __future__ import annotations\n- from typing import
List, Optional, Tuple\n- from dataclasses import dataclass\n- # Window-level audio cue features
(order = persisted into candidate_segments.stats)\n- AUDIO_FEATURE_NAMES = [\n
"audio_hit_count", # raw count of detected hits in window\n "audio_hit_rate",
# hits per second (fps-invariant)\n "audio_hit_strength_mean", # mean onset-envelope peak
(arbitrary units)\n]\n- \n- def extract_hits_in_window(hits: List[Any], window_start: float,
window_end: float) -> List[Any]:\n- "\n- "\n- Filter hits to those with t in [window_start,
window_end).\n- "\n- "\n- return [h for h in hits if window_start <= h.t < window_end]\n- \n- def
compute_audio_features(hits: List[Any], window_start: float, window_end: float) -> Dict[str,
float]:\n- "\n- "\n- Compute the ~3 audio cue features from raw hit events.\n- \n- Args:\n
hits: list of HitEvent from detect_hits_in_video()\n- window_start, window_end: seconds
(the candidate window bounds)\n- \n- Returns: dict with keys AUDIO_FEATURE_NAMES; all values
float, default 0.0 on empty.\n- "\n- "\n- win_hits = extract_hits_in_window(hits,
window_start, window_end)\n- win_duration = max(1e-6, window_end - window_start)\n- \n-
strength_vals = [h.strength for h in win_hits]\n- mean_strength = (sum(strength_vals) /
len(strength_vals)) if strength_vals else 0.0\n- \n- return {\n- "audio_hit_count":
float(len(win_hits)),\n- "audio_hit_rate": float(len(win_hits)) / win_duration,\n-
"audio_hit_strength_mean": float(mean_strength),\n- }\n- ``\n\n--\n\n#### 1.3
FusionSegmenter Wiring (UPDATED)\n\n**File:** `backend/pipeline/segmenters/fusion_hybrid.py`
\n\n**Change:** Extend `_enrich_candidate_stats()` with audio cue branch (same pattern as person
cue)\n\n**Location:** Lines 76?90 (current `_enrich_candidate_stats`)\n\n**New

```

```

Wiring:**\n``python\ndef _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any],
fps: float,\n                                frame_width: float, video_path: str,\nidx_cfg: Dict[str, Any]) -> Dict[str, Any]:\n    stats = super()._enrich_candidate_stats(cw,
points, fps, frame_width, video_path, idx_cfg)\n    \n    # Gate-A signal ? net-crossing count
(EXISTING, lines 80?85)\n    gated = rally_gate.gate_windows(points, [(cw["start"],
cw["end"])]), fps,\n                                min_crossings=0,
estimate=self._axis_estimate(points))\n    stats["net_crossings"] = float(gated[0].crossings)
if gated else 0.0\n    \n    # Person-cue features (EXISTING, lines 87?90)\n    fcfg =
idx_cfg.get("fusion", {})\n    if fcfg.get("cue_flags", {}).get("person", False):\n
stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))\n    \n    #
Audio-cue features (NEW, SAME PATTERN)\n    if fcfg.get("cue_flags", {}).get("audio",
False):\n        stats.update(self._audio_features(cw, fps, video_path, fcfg))\n    \n    return
stats\n\n``\n\n**New Private Method (add to FusionSegmenter class after
`_person_features`):**\n``python\ndef _audio_features(self, cw: Dict[str, Any], fps: float,
video_path: str,\n                                fcfg: Dict[str, Any]) -> Dict[str, float]:\n    \n    \n    GPU-
free path (audio cue ON): detect hits in this window's video timespan,\n    aggregate into
scale-invariant features, and persist. Lazily builds ONE audio\n    detector per video (cached).
Pure Python; no torchvision imports.\n    \n    \n    from backend.pipeline.audio.hit_detector
import detect_hits_in_video\n    from backend.pipeline.detectors.audio_features import
compute_audio_features\n    \n    if not hasattr(self, '_audio_hits'):\n        \n    # Lazy: run hit
detection once per video, cache the full hit list.\n    \n    # Config knobs: k (threshold), band
(Hz tuple), min_gap_sec, sr (sample rate)\n    audio_cfg = fcfg.get("audio_params", {})\n
self._audio_hits = detect_hits_in_video(\n        \n        video_path,\n        sr=int(audio_cfg.get("sample_rate", 16000)),\n
k=float(audio_cfg.get("threshold_k", 2.5)),\n        \n        band=tuple(audio_cfg.get("band"))
if audio_cfg.get("band") else None,\n        min_gap_sec=float(audio_cfg.get("min_gap_sec", 0.08)),\n        \n        \n    )\n    \n    return
compute_audio_features(self._audio_hits, cw["start"], cw["end"])\n\n``\n\n**Config Schema
(under `indexing.fusion`):**\n``yaml\nfusion:\n  substrate: "wasb" # or "tracknet"\n  cue_flags:\n    person: false # EXISTING\n    audio: false # NEW, default-OFF\n  \n  #
Optional audio tuning knobs (used only if audio cue enabled)\n  audio_params:\n    sample_rate:
16000 # Hz (default)\n    threshold_k: 2.5 # sensitivity knob (default; higher =
fewer hits)\n    band: null # null = no band limit; [3000, 9000] = HF band\n
min_gap_sec: 0.08 # minimum inter-hit spacing (seconds)\n\n``\n\n---\n\n### 2. Persisted
Features Path\n\n**Database Location:** `candidate_segments.stats` JSON column (EXISTING
pattern) \n\n**Lifetime:** Per candidate window; persisted once, never recomputed\n\n**Feature
Columns Added (when `fusion.cue_flags.audio = true`):**\n\n``json\n{\n  "duration": 2.5,\n
"peak_velocity": 0.12,\n  "mean_velocity": 0.08,\n  "active_frame_count": 150,\n
"track_id_count": 2,\n  "net_crossings": 3.0,\n  "audio_hit_count": 8.0,\n
"audio_hit_rate": 3.2,\n  "audio_hit_strength_mean": 0.45\n}\n\n``\n\n**Feature Order in
Experiment Harness:**\n\n- Shuttle features (EXISTING, distill_local line 40): duration,
peak_velocity, mean_velocity, active_ratio, net_crossings\n\n- Person features (if enabled,
fusion_features line 194): players_in_court, two_sided_motion, ... \n\n- **Audio features (if
enabled, NEW):** audio_hit_count, audio_hit_rate, audio_hit_strength_mean\n\n---\n\n### 3.
Experiment Harness Integration\n\n**File:** `backend/eval/experiment.py` \n\n**Pattern:** Audio
cue is a *variant*, not a code branch. The `Variant` dataclass (line 74) carries `cue_flags`
(line 107); when audio is in that dict, the framework passes it through `load_video_candidates`
? `_feature_column`.\n\n**Usage Example:**\n\n``python\nfrom backend.eval.experiment import
Variant, compare\n\n# Control (no audio)\ncontrol = Variant(name="baseline",
segmenter="wasb_hybrid")\n\n# Audio ON, features in the learned filter\naudio_recipe =
Variant(\n    name="wasb+audio",\n    segmenter="fusion_hybrid",\n    cue_flags={"audio":
True},\n    config_overrides={"fusion": {"audio_params": {"threshold_k": 2.5}}},\n
feature_names=["duration", "peak_velocity", "mean_velocity", "active_ratio",\n
"net_crossings", "audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"],\n
threshold=0.5,\n    merge_gap=1.0,\n)\n\n# Offline A/B on labeled corpus\nresult =
compare([video1, video2], control, audio_recipe, iou=0.5, honest=True)\n\n``\n\n**Honest LOVO:**
Per fold, `compare()` trains on OTHER videos' audio features (if present), predicts the held-out
one. Audio being weak (~2.2x) means the learned weights will likely be small; the leaderboard
will show if audio helps or hurts (expected: marginal, not harmful if it's truly just ~0
weight).\n\n---\n\n### 4. Graceful Degradation & Robustness\n\n**Audio Detection Failure:**\n\n-
ffmpeg missing ? `extract_audio` raises (caught by caller, logs error)\n\n- Corrupted/no audio ?
`load_wav` returns silence (all-zeros); `onset_envelope` returns empty array; `detect_hits`
returns `[]`\n\n- Empty hits list ? `compute_audio_features` returns all-0.0 (no NaN, no
exception)\n\n**Backwards Compatibility:**\n\n- Default: `fusion.cue_flags.audio = false` ? no
audio detection, no stats columns added, `FusionSegmenter` is byte-identical to today\n\n- Old

```

persisted candidates (before audio feature addition) ? `load\_video\_candidates` maps missing columns to 0.0 (line 176 of experiment.py)\n\n**Unit Test Points:**\n-  
`test\_compute\_audio\_features` (empty/one-hit/many-hit windows)\n-  
`test\_audio\_features\_in\_fusion\_segmenter` (mock `detect\_hits\_in\_video`, verify stats added when cue ON, absent when OFF)\n-  
`test\_experiment\_with\_audio\_features` (load candidate with audio columns, train/predict variant)\n\n---\n\n### 5. File Locations & Line References (Exact)\n\n| File | Lines | Change |\n|-----|-----|-----|\n| `backend/pipeline/audio/hit\_detector.py` | 23?26, 136?146 | **EXISTING** ? `HitEvent`, `detect\_hits\_in\_video()` |\n| `backend/pipeline/detectors/audio\_features.py` | ? | **NEW** ? `AUDIO\_FEATURE\_NAMES`, `compute\_audio\_features()` |\n| `backend/pipeline/segmenters/fusion\_hybrid.py` | 76?90 (old), 76?103 (new) | **UPDATED** ? add audio cue branch in `\_enrich\_candidate\_stats` + new `\_audio\_features()` method |\n| `backend/eval/distill\_local.py` | 40 | **UNCHANGED** ? `FEATURE\_NAMES` stays as-is (experiment harness extends it per variant) |\n| `backend/eval/experiment.py` | 74?114 (Variant dataclass) | **UNCHANGED** ? already supports arbitrary `cue\_flags` and `feature\_names` |\n| `backend/eval/experiment.py` | 429?447 | **UNCHANGED** ? `load\_video\_candidates()` already maps missing columns ? 0.0 |\n| `tests/test\_audio\_features.py` | ? | **NEW** ? unit tests for `compute\_audio\_features()` |\n| `tests/test\_fusion\_hybrid.py` | ? | **UPDATED** ? add test case for audio cue ON\n\n---\n\n### 6. Configuration Example (indexing.json)\n\n```json\n{\n \"indexing\": {\n \"segmenter\": \"fusion\_hybrid\", \n \"fusion\": {\n \"substrate\": \"wasb\", \n \"cue\_flags\": {\n \"person\": false, \n \"audio\": false \n }, \n \"audio\_params\": {\n \"sample\_rate\": 16000, \n \"threshold\_k\": 2.5, \n \"band\": null, \n \"min\_gap\_sec\": 0.08 \n }, \n \"min\_crossings\": 0, \n \"min\_players\": 0, \n \"court\_polygon\": null, \n \"net\_axis\": \"y\", \n \"net\_position\": 0.5 \n } \n } \n}\n```\n\n---\n\n### 7. Expected Outcome & Honest Assessment\n\n**Per E1 findings (AUDIO\_RALLY\_DETECTION\_PLAN lines 95?120):**\n- Audio hit-clusters ARE present in rallies (recall signal real)\n- Discrimination ceiling ~2.2x (in-rally vs dead-time) ? toss/footwork noise is indistinguishable\n- Band-pass and classical onset methods do NOT break through\n- **What This Design Does:**\n- Persists the weak signal (3 features per window) without forcing adoption\n- Lets the learned scorer (LogisticRegression on LOVO-labeled data) assign its own weights\n- Provides the **exact** same offline experimental workflow as person (A/B via `compare()`)\n- Leaves the decision to the golden-label corpus: if 2.2x is too weak, weights ? ~0; if there's hidden structure the classifier finds, it lights up\n- **Honest Expectation:**\n- Probability: learned weights ? 0 (cannot overcome discrimination ceiling)\n- Downside risk: negligible (weak signal, small feature column)\n- Upside: if E2 (learned timbre classifier) lands, this plumbing is the reuse layer\n- Path to next signal: audio framework is in place; next lever = E2 or pose/stance (parked in ADR-007)\n\n---\n\n### Summary Table\n\n| Aspect | Detail |\n|-----|-----|\n| **Cue Status** | Default-OFF (`fusion.cue\_flags.audio` = false) |\n| **Detection** | Existing `hit\_detector.py` (classical onset, no learning) |\n| **Features** | 3 per window: hit\_count, hit\_rate, mean\_strength |\n| **Persistence** | Per-candidate JSON in `candidate\_segments.stats` |\n| **Gating** | None (no decision gate; decision purely in learned filter) |\n| **Experiment** | Variant with `feature\_names` including audio columns, scored LOVO-honestly |\n| **Config** | `indexing.fusion.cue\_flags.audio`, `indexing.fusion.audio\_params` |\n| **Test Pattern** | Mock `detect\_hits\_in\_video`, verify stats add/drop with cue flag |\n| **Rollback** | Flip `fusion.cue\_flags.audio` to false; no code changes needed |\n| **anchors** | [\"backend/pipeline/audio/hit\_detector.py:23-26 (HitEvent dataclass)\", \"backend/pipeline/audio/hit\_detector.py:136-146 (detect\_hits\_in\_video)\", \"backend/pipeline/segmenters/fusion\_hybrid.py:39 (FusionSegmenter class)\", \"backend/pipeline/segmenters/fusion\_hybrid.py:76-90 (\_enrich\_candidate\_stats method, current)\", \"backend/pipeline/detectors/fusion\_features.py:193-197 (PERSON\_FEATURE\_NAMES pattern)\", \"backend/eval/experiment.py:74-114 (Variant dataclass)\", \"backend/eval/experiment.py:429-447 (load\_video\_candidates)\", \"backend/eval/distill\_local.py:40 (FEATURE\_NAMES)\"] |\n| **findings** | **GPU-RUN RECIPE FOR P3 ON RTX 2070 SUPER + WSL2**\n\n### COST & TIME ANALYSIS (6 Golden Videos)\n- **Total workload:**\n- Input: 208,421 raw frames @ 29.97?59.94 fps across 6 golden videos (37.1GB, 256 rallies)\n- Processing: **69,473 sampled frames** (with `frame\_skip=3` from config.json line 36)\n- Model: `fasterrcnn\_resnet50\_fpn` (41.8M params, 160MB weights, BSD license COCO weights)\n- Output: ~250?280 candidate windows persisted to DB\n- `candidate\_segments.stats` with 5 person features each\n- **Time estimate (RTX 2070 Super @ 8GB VRAM):**\n- Per-frame inference: **50ms average** (fasterrcnn\_resnet50\_fpn + ByteTrack overhead, varies 40?60ms with resolution 1080p?5.3K)\n- 69,473 frames x 50ms = **57.9 minutes pure inference**\n- **Practical wall-clock: 90?120 minutes** (with I/O, cv2 frame seeks, tracker state, Python overhead)\n- **Recommended schedule: 2 hours headless, no user interaction**\n- **Memory profile (GPU):**\n- Model weights: 160MB\n- Activations + ByteTrack history (per 1-frame batch): ~350MB\n- Total resident: ~600MB; **7.4GB free margin** ? very safe

```

for 8GB VRAM\n- No OOM risk even with larger input frames (5.3K resolution)\n\n--\n\n### ACTUAL
FILE LOCATIONS & ENTRYPOINTS\n\n**Golden trajectory CSVs (EXISTING ? NO WASB RE-RUN
NEEDED):**\n- Trajectories cached in `output/wasb/<video_stem>_<md5>_wasb_wasbcache/`
directories\n- CSV files: `output/wasb/GX010125_1080p__89605e80ed01_wasb.csv`,
`output/wasb/mahadevpura_smoke_180s__05e0e577029d_wasb.csv`\n- These are ALREADY ingested;
person detection runs OVER these existing shuttle points\n\n**Person detector
implementation:**\n- `backend/pipeline/detectors/person_detector.py:258?303` defines
`detections_per_frame(video_path, start_frame, end_frame, frame_skip)`\n- Called by
`FusionSegmenter._person_features()` at
`backend/pipeline/segmenters/fusion_hybrid.py:103?131`\n- Lazy model load at
`person_detector.py:65?89` (no GPU touched until first detection)\n\n**Database schema:**\n-
Candidate windows stored in `candidate_segments` table\n- Person features persisted to `stats`
JSON column (5 keys: `players_in_court`, `two_sided_motion`, `mean_player_motion`,
`in_court_ratio`, `shuttle_player_colocation`)\n- See `fusion_features.py:192?216` for
`PERSON_FEATURE_NAMES` and `compute_person_features()` signature\n\n**Manifest & experiment
harness:**\n- Golden videos manifest: `C:\\Users\\avidu\\Projects\\sports-data-
collector\\data\\eval_export\\manifest.json` (line 27?100 lists 6 videos)\n- Experiment runner:
`backend/eval/experiment.py:150?300` enables A/B via `Variant` +
`compare()`\n- Key entry: `Variant.is_learned()` (line 115?117),
`load_video_candidates()` bridges DB to experiment layer\n\n--\n\n### HOW TO RUN P3 PERSON
DETECTION (LIGHTWEIGHT OFFLINE PATH)\n\n**Option 1: FusionSegmenter with person cue ON
(recommended for offline fine-tuning)**\n\n``bash\n# PowerShell\n$env:GEMINI_API_KEY = \"your-
key-here\" # Set once; P3 uses Gemini for semantic boundary refinement\n\npython -m
backend.main \\\n    --video-path \"C:\\Users\\avidu\\Projects\\Annotation
Setup\\Collect\\Golden Labelled\\Adarsh and Avi.MP4\" \\\n    --output-dir output \\\n
--segmenter fusion_hybrid\n\n``\n\n**To enable person cue in config.json (lines 55?66
currently):**\n\n``json\n\"indexing\": {\n  \"detector_model\": \"fasterrcnn_resnet50_fpn\", \n
\"detector_score_threshold\": 0.5, \n  \"yolo_frame_skip\": 3, \n  \"wasb\": {\n
\"velocity_threshold\": 0.02 }, \n  \"fusion\": {\n    \"substrate\": \"wasb\", \n
\"cue_flags\": { \"person\": true }, \n    \"min_crossings\": 0, \n    \"min_players\": 0\n
}\n}\n\n``\n\n**Option 2: Batch run (all 6 golden videos) via
`run_phase3_eval.py`**\n\n``bash\npython run_phase3_eval.py \\\n    --manifest
C:\\Users\\avidu\\Projects\\sports-data-collector\\data\\eval_export\\manifest.json \\\n
--segmenter fusion_hybrid \\\n    --videos-root \"C:\\Users\\avidu\\Projects\\Annotation
Setup\\Collect\\Golden Labelled\" \\\n    --stage both\n\n``\n\nThis:\n- Runs FusionSegmenter on
each video (person detection ? candidate windows ? Gemini handoff)\n- Scores against golden CSV
labels\n- Outputs per-video + corpus metrics to `output/phase3_report.json`\n\n--\n\n###
HEADLESS + KEEP-AWAKE (MODERN STANDBY MITIGATION)\n\n**Problem:** Windows 11 Pro's \"Modern
Standby\" (S0i3) can interrupt long GPU runs even with `powercfg` High Performance mode
active.\n\n**Solution (no code changes needed ? external scripting):**\n\nCreate
`keep_aware.ps1` (PowerShell):\n\n``powershell\n# Keep display/GPU active for long-running
detection\n$wshell = New-Object -ComObject wscript.shell\nwhile ($true) {\n  # Simulate key
press every 2 minutes (Windows interprets as user activity)\n  $wshell.SendKeys(\"{NUMLOCK}\")\n  Start-Sleep -Seconds 120\n}\n\n``\n\nRun P3 in WSL2 Ubuntu
(detached from Windows scheduler):\n\n``bash\n# In WSL2 Ubuntu bash\nnohup python
run_phase3_eval.py \\\n    --manifest /mnt/c/Users/avidu/Projects/sports-data-
collector/data/eval_export/manifest.json \\\n    --segmenter fusion_hybrid > /tmp/p3.log 2>&1
&\n\n# Monitor from Windows PowerShell (check every 5 min)\nwhile ($true) { tail -f
/mnt/c/Users/avidu/Projects/badminton-highlight-indexer/output/phase3_report.json; Start-Sleep
300 }\n\n``\n\n**Alternative: System-level disable Modern Standby (Windows):**\n\n``powershell\n#
PowerShell (Admin)\npowercfg /change monitor-timeout-ac 0 # Never turn off monitor on
AC\npowercfg /change disk-timeout-ac 0 # Never turn off disk on AC\npowercfg
/setacvalueindex scheme_current sub_sleep standbystate 0 # Disable standby entirely\npowercfg
/setactive scheme_current\n\n``\n\n**WSL2-specific:** GPU calls from Ubuntu directly don't
trigger Modern Standby, but the Windows host can. Run person detection *inside* the WSL2 conda
environment if using TrackNet/WASB, then copy results back.\n\n--\n\n### CACHE & OUTPUT
HYGIENE\n\n**Before P3 run (clean slate):**\n\n``bash\n# Remove old candidate/feature caches
(safe; WASB trajectory remains)\nrm -rf output/wasb/*_wasbcache\nrm output/sports_indexer.db #
Or use --reprocess to skip existing videos\n\n# Keep WASB trajectory CSVs (they're your golden
trajectory, reused by person detector)\nls output/wasb/*_wasb.csv # Should exist; don't
delete\n\n``\n\n**During run (automatic cleanup per CODE_MAP $2.2 line 127):**\n- WasbRunner (if
used) deletes intermediate frames + staged video on success\n- TrackNet (if used) deletes staged
video\n- Controlled by `indexing.wasb.keep_frames` (default `false`; keep if run fails for
resume)\n\n**After run (archive results):**\n\n``bash\n# Persist the feature-enriched candidate
DB for downstream analysis\nncp output/sports_indexer.db

```



```

output/sports_indexer_p3_person_v1.db\n\n# Export candidate segments for experiment harness
(used by eval/experiment.py)\npython -c \"\nfrom backend.infrastructure.database import
Database\nndb = Database('output/sports_indexer.db')\ncandidates =
db.query_candidate_segments()\nprint(f'Persisted {len(candidates)} candidates with person
features')\n\n\n\n# CSV for manual review\nsqlite3 output/sports_indexer.db \".mode csv\" \"\n
\"SELECT id, video_id, detector, stats FROM candidate_segments LIMIT 10\" >
/tmp/candidates_sample.csv\n\n\n\n\n\n\n### MONITORING PROGRESS\n\n**Real-time (while
running):**\n\n\n`bash\n# Terminal 1: Watch stderr logs (model load + per-frame progress)\ntail
-f output/phase3.log | grep -E \"Loading|Processing|detected|window\"\n\n\n# Terminal 2: Monitor
GPU/memory\n# PowerShell: wsl -d Ubuntu nvidia-smi -l 1 (updates every sec)\n# Or WSL bash:
watch -n1 nvidia-smi\n\n\n# Terminal 3: Poll database for candidate count\nwhile true; do\n
count=$(sqlite3 output/sports_indexer.db \"SELECT COUNT(*) FROM candidate_segments;\")\n echo
\"Candidates so far: $count\"\n sleep 30\ndone\n\n\n\n**Post-run verification:**\n\n`python\n#
Python: check person features were persisted\nimport json\nfrom backend.infrastructure.database
import Database\nndb = Database(\"output/sports_indexer.db\")\nfor cand in
db.query_candidate_segments():\n    stats = json.loads(cand.get(\"stats\", \"{}\"))\n
person_keys = [k for k in stats if k.startswith((\"players\", \"shuttle\", \"two_sided\",
\"in_court\", \"mean\"))]\n    if person_keys:\n        print(f\"Video {cand['video_id']}:
{len(person_keys)} person features ?\")\n        break\n\n\n\n\n\n\n### LIGHTWEIGHT OFFLINE
PERSON-FEATURE EXTRACTOR (PREFERRED) VS FUSIONSEGMENTER\n\n**Lightweight path (experimental
harness only):**\n\n\n`bash\n# Extract person features offline without re-running trajectory
detection\n# or Gemini handoff ? pure local CPU math (scores persisted candidates)\npython -c
\"\nfrom backend.pipeline.detectors.person_detector import PersonDetector\nfrom
backend.pipeline.detectors.fusion_features import compute_person_features\nfrom
backend.infrastructure.database import Database\nncfg = {'indexing': {'use_gpu':
True}}\ndetector = PersonDetector(cfg)\nndb = Database('output/sports_indexer.db')\nfor cand in
db.query_candidate_segments():\n    video_id, start, end = cand['video_id'], cand['start_time'],
cand['end_time']\n    # Get the ORIGINAL video path from the manifest\n    det =
detector.detections_per_frame(video_path, int(start * fps), int(end * fps), frame_skip=3)\n
features = compute_person_features(det['frames'], trajectory_points, ...)\n    # Update
candidate_segments.stats without re-running WASB\n\n\n\n\n\n**Full FusionSegmenter
(production):**\n- Runs WASB trajectory detection (GPU, WSL2, 2?5 hrs)\n- Computes person
features (GPU, 1?2 hrs) ? **YOU ARE HERE**\n- P3 handoff to Gemini (API calls, ~5 min)\n-
Result: play_segments (semantic boundaries)\n\n\n**Decision rule:**\n- **Use lightweight if:** You
have existing golden WASB trajectories + want to tune person-feature thresholds offline\n- **Use
FusionSegmenter if:** You need end-to-end (no WASB trajectories yet) or want semantic refinement
from Gemini\n\n\nFor your 6 golden videos: **lightweight preferred** because trajectories already
exist. The person-feature computation is *replayable* ? run it once, then experiment with
different gate thresholds (`min_players`, `min_crossings`) without re-detecting.\n\n\n\n\n###
SAMPLE INVOCATION (END-TO-END P3)\n\n\n`bash\n# 1. Ensure config.json has person cue enabled +
GEMINI_API_KEY set\n$env:GEMINI_API_KEY = \"your-gemini-api-key\"\n\n\n# 2. Run batch phase3 with
person detection\npython run_phase3_eval.py \"\" --manifest
C:\\Users\\avidu\\Projects\\sports-data-collector\\data\\eval_export\\manifest.json \"\n\n
--segmenter fusion_hybrid \"\" --videos-root \"C:\\Users\\avidu\\Projects\\Annotation
Setup\\Collect\\Golden Labelled\" \"\n\n --stage both \"\" --json
output/phase3_person_report.json \"\n\n --show-failures\n\n\n# 3. Archive results\nncp
output/sports_indexer.db output/sports_indexer_p3_golden_${date +%Y%m%d}.db\n\n\n# 4. Run
experiment harness to compare vs control (no person cue)\npython -c \"\nfrom
backend.eval.experiment import Variant, compare, CONTROL\nfrom backend.infrastructure.database
import Database\nndb = Database('output/sports_indexer.db')\nncfg = {}\n\n\nperson_variant =
Variant(\n    name='person_cue_v1',\n    segmenter='fusion_hybrid',\n
config_overrides={'indexing.fusion.cue_flags.person': True},\n    min_players=2\n)\n\n\nresult =
compare(CONTROL, person_variant, db, cfg)\nprint(f'Person cue impact:
{result}')\n\n\n\n\n\n\n\n### REALISTIC EXPECTATIONS & GOTCHAS\n\n1. **Frame-seek accuracy
(cv2 codec dependency):** Person detection uses `cv2.CAP_PROP_POS_FRAMES` to seek to candidate-
window boundaries. On H.264/H.265, seek lands  $\pm$  a few frames from keyframes. Features computed on
\"wrong\" frames (off by 1?3 frames) remain valid ? they're aggregate ratios, not per-frame
labels. No action needed, but know the alignment is ~frame-level, not sub-frame.\n\n2.
**ByteTrack chunk-local IDs:** PersonDetector creates a fresh ByteTrack per window
(`make_tracker` line 91?98). Track IDs don't persist across window boundaries. This is correct ?
features are computed within-window (e.g., \"median players in court\"), not across.\n\n3.
**Gemini API cost P3:** Even with person features pre-computed, FusionSegmenter still calls
Gemini for semantic boundary refinement. If you want *pure* offline (no API), use
`experiment.py`'s `compare()` with only motion/net-crossing gates and skip the provider
handoff.\n\n4. **Multi-court confound (known limitation):** gBaaddy + Kushagra videos have

```

background courts visible. WASB trajectory includes both foreground + background shuttles (it's \"track everything\"). Person features (in-court counts, two-sided motion) will be inflated by background players. The learned fusion (distill\_local) learns to downweight these features in multi-court footage ? no immediate fix but logged in

```
eval_baselines/experiment_leaderboard.json.\n\n5. **GPU memory headroom:** 8GB is safe for
single-frame batching (no batch accumulation). If you later add larger models or concurrent
batch inference, monitor `nvidia-smi` ? the 600MB resident should leave 7.4GB for
spikes.\n\n---\n\n### REFERENCES (FILE:LINE FOR PRECISION)\n\n- Person detector definition:
`backend/pipeline/detectors/person_detector.py:50?303`\n - `detections_per_frame()`: line
258?303 (ORIGINAL-video frame indexing for fusion)\n - Model loading: line 65?89 (lazy,
torchvision)\n - `_PERSON_CLASS_ID = 1`: line 38 (COCO class, NOT ultralytics' 0)\n \n-
FusionSegmenter orchestration: `backend/pipeline/segmenters/fusion_hybrid.py:39?154`\n -
`_person_features()` GPU path: line 103?131\n - Gate A/B application: line 133?151\n \n-
Feature computation (pure, GPU-free): `backend/pipeline/detectors/fusion_features.py:192?216`\n
- `PERSON_FEATURE_NAMES`: line 194?197\n - `compute_person_features()`: line 200?215\n \n-
Experiment harness (A/B comparison): `backend/eval/experiment.py:1?150`\n - `Variant`
dataclass: line 73?143\n - `compare()` & `compare_unlabeled()`: signatures in docstring, line
25?31\n \n- Config schema (defaults): `backend/config/models.py`\n -
`IndexingConfig.yolo_frame_skip`: default 3 (person detector inherits)\n -
`IndexingConfig.detector_model`: default `fasterrcnn_resnet50_fpn`\n \n- Database schema:
`backend/infrastructure/database.py::add_candidate_segment()`\n - `stats` JSON column holds
person feature dict (persisted, immutable)\n \n- Run scripts: \n - Single video:
`run_process_and_stitch.py` (interactive, skip this)\n - Batch phase3:
`run_phase3_eval.py:75?150` (manifest walk + segment + score)\n - Experiment:
`backend/eval/experiment.py` + `run_regression.py` (comparison gate)","anchors":["backend/pipeli
ne/detectors/person_detector.py:258?303","backend/pipeline/segmenters/fusion_hybrid.py:103?131",
"backend/pipeline/detectors/fusion_features.py:192?216","backend/eval/experiment.py:73?143","con
fig.json:33?66","C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden Labelled (6
videos, 208,421 raw frames)","output/wasb/*_wasb.csv (existing golden trajectories)","RTX 2070
Super: 50ms/frame x 69,473 sampled frames = 58?120 min wall-clock","Modern Standby mitigation:
keep_awake.ps1 or powercfg /change"]}]`.
```

Constraints: reuse the shipped FusionSegmenter / fusion\_features / PersonDetector.detections\_per\_frame / experiment harness ? no reinvention. NO special-casing: audio + person enter as confidence-scored FEATURES the learned scorer weights, measured via the harness before any promotion; everything default-OFF; LOVO-honest (train on other videos, score held-out). The GPU person-feature extraction for the golden videos must reuse the EXISTING trajectory CSVs (no WASB re-run). Maximize what's a clean tracked PR vs the GPU run that produces the number.

Output: (1) plan narrative; (2) p3\_steps (ordered, incl. which is a tracked code PR vs the GPU measurement run); (3) p4\_steps; (4) gpu\_run (the exact run recipe + keep-awake + monitoring); (5) files to add/edit; (6) open questions. Prefer a small offline eval driver for the golden person-feature extraction + harness LOVO over running the full Gemini pipeline.

## 2. assistant (2026-06-13T20:00:07.147Z)

I'll synthesize a concrete plan from these maps. Let me first verify the key anchors on master so the plan reflects real shipped state, not the maps' assumptions.

## 3. user (2026-06-13T20:00:09.917Z)

Already up to date.

## 4. assistant (2026-06-13T20:00:12.572Z)

Let me verify the key anchors actually exist on master before committing to a plan built on them.

## 5. user (2026-06-13T20:00:14.207Z)

```
backend/eval/:
__init__.py
__pycache__
```

```
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
experiment.py
export_wasb_segments.py
gemini_refine.py
gt_loader.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py
```

```
backend/pipeline/audio/:
__init__.py
__pycache__
hit_detector.py
```

```
backend/pipeline/detectors/:
__init__.py
__pycache__
base.py
fusion_features.py
person_detector.py
rally_gate.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py
```

```
backend/pipeline/segmenters/:
__init__.py
__pycache__
base.py
fusion_hybrid.py
gemini.py
motion.py
tracknet_hybrid.py
trajectory_hybrid.py
wasb_hybrid.py
yolo_hybrid.py
```

## 6. user (2026-06-13T20:00:15.943Z)

```
output/human_lovo_manifest.json
output/human_lovo_manifest.json
output/kushagra_1280.report.json
output/local_calib_manifest.json
output/local_rally_model.json
output/mahadevpura_smoke_180s.report.json
output/phase3_probe.json
run_phase3_eval.py
---WASB CSV---
output/wasb/GX010125_1080p__89605e80ed01_wasb.csv
output/wasb/mahadevpura_smoke_180s__05e0e577029d_wasb.csv
```

## 7. assistant (2026-06-13T20:00:19.235Z)

Two important findings already: only 2 WASB CSVs exist on master (not 6), and there's a `backend/eval/audio` dir. Let me dig into the critical files ? experiment.py's actual API

surface (compare, evaluate\_variant, load\_video\_candidates, Variant), the manifest, and fusion\_features/audio.

## 8. user (2026-06-13T20:00:19.760Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. **No new scoring math lives
7      here**; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54
55     logger = logging.getLogger(__name__)
56
57     # Where a comparison run appends one reproducible record. Tracked location (NOT under
58     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
59     # regression.DEFAULT_BASELINE_PATH.
60     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
```

```

61 _METRICS = ("precision", "recall", "f1", "mean_iou")
62
63
64 class ExperimentError(ValueError):
65     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
66     score an unlabeled video with no pinned model, or a gate referencing an absent
67     feature)."""
68
69
70 # ----- Variant
71
72
73 @dataclass
74 class Variant:
75     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
76
77     A variant turns one video's persisted candidate windows + features into rally
78     intervals. Every knob defaults to the control behaviour (no gate, no learned
79     filter), so a variant that merely carries a new, disabled cue is byte-identical
80     to control ? the core no-regression guarantee.
81
82     Fields:
83     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
84       for the leaderboard and for selecting the served path (the existing
85       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
86       New cues are recorded here default-OFF.
87     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
88       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
89       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
90       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
91     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
92       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
93       cue persists that feature).
94     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
95       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
96       required for ``compare_unlabeled`` on a learned variant.
97     - ``feature_names`` ? the persisted features the learned filter consumes. When set
98       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
99       (honest) ? the recipe, not a pinned artifact.
100     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
101       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
102     """
103
104     name: str
105     segmenter: str = "wasb_hybrid"
106     config_overrides: Dict[str, Any] = field(default_factory=dict)
107     cue_flags: Dict[str, bool] = field(default_factory=dict)
108     min_crossings: int = 0
109     min_players: int = 0
110     classifier_model: Optional[str] = None
111     feature_names: Optional[List[str]] = None
112     threshold: float = 0.5
113     merge_gap: float = 1.0
114
115     def is_learned(self) -> bool:
116         """True if this variant applies a learned classifier (pinned model or
recipe)."""
117         return self.classifier_model is not None or bool(self.feature_names)
118
119     def to_dict(self) -> dict:
120         return {
121             "name": self.name,
122             "segmenter": self.segmenter,
123             "config_overrides": self.config_overrides,

```

```

124         "cue_flags": self.cue_flags,
125         "min_crossings": self.min_crossings,
126         "min_players": self.min_players,
127         "classifier_model": self.classifier_model,
128         "feature_names": self.feature_names,
129         "threshold": self.threshold,
130         "merge_gap": self.merge_gap,
131     }
132
133     @classmethod
134     def from_dict(cls, d: dict) -> "Variant":
135         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
136         return cls(**{k: v for k, v in d.items() if k in known})
137
138     def spec_hash(self) -> str:
139         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
140         and makes a result reproducible. The pinned **control** variant always hashes
141         the
142         same, so a regression is always traceable to a recipe change, never to drift."""
143         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
144         return sha1(blob.encode()).hexdigest()[:12]
145
146 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
147 #: filter. Always the comparison reference; "rollback" = make this the served variant.
148 CONTROL = Variant(name="control")
149
150
151 # ----- persisted candidates
152
153
154 @dataclass
155 class VideoCandidates:
156     """One video's persisted detection, ready for offline re-scoring.
157
158     ``intervals`` are the candidate windows; ``features`` is column-major
159     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
160     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
161     with ``load_video_candidates``, or directly in tests.
162     """
163
164     name: str
165     intervals: List[Interval]
166     features: Dict[str, List[float]] = field(default_factory=dict)
167     gts: Optional[List[Interval]] = None
168
169
170     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
171         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
172         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
173         col = vc.features.get(name)
174         n = len(vc.intervals)
175         if col is None:
176             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
177                             name, vc.name)
178             return [0.0] * n
179         if len(col) != n:
180             raise ExperimentError(
181                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
182         return [float(x) for x in col]
183
184
185     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
186     List[List[float]]:
187         cols = [_feature_column(vc, name) for name in feature_names]

```

```

187         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
188
189
190     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
191         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
192         players)."""
193         n = len(vc.intervals)
194         mask = [True] * n
195         if variant.min_crossings > 0:
196             col = _feature_column(vc, "net_crossings")
197             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
198         if variant.min_players > 0:
199             col = _feature_column(vc, "players_in_court")
200             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
201         return mask
202
203     def predict(variant: Variant, vc: VideoCandidates,
204                 model: Optional[LogisticRegression] = None) -> List[Interval]:
205         """Variant prediction: gate by persisted features, optionally filter by a learned
206         model, then merge. Pure and deterministic.
207
208         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
209         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
210         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
211         is nothing to score with.
212         """
213         mask = _gate_mask(variant, vc)
214         if variant.feature_names:
215             if model is None:
216                 if variant.classifier_model is None:
217                     raise ExperimentError(
218                         f"variant {variant.name!r} is learned but has no model to predict "
219                         f"with (pass a fitted model or set classifier_model)")
220                 model = LogisticRegression.load(variant.classifier_model)
221                 proba = model.predict_proba(_feature_matrix(vc, variant.feature_names))
222                 mask = [m and (float(proba[i]) >= variant.threshold) for i, m in
223                         enumerate(mask)]
224             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
225             return merge_overlaps(kept, gap_tol=variant.merge_gap)
226
227     # ----- variant scoring
228
229
230     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
231     LogisticRegression:
232         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
233         the
234         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
235         X: List[List[float]] = []
236         y: List[int] = []
237         for vc in videos:
238             if vc.gts is None:
239                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
240             X.extend(_feature_matrix(vc, variant.feature_names or []))
241             y.extend(label_candidates(vc.intervals, vc.gts, iou))
242         if not X:
243             raise ExperimentError("cannot train a learned variant on zero candidates")
244         return LogisticRegression().fit(X, y, variant.feature_names)
245
246     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
247                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
248         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a

```

```

248         mean aggregate.
249
250     Prediction policy:
251     - **static variant** (no learned filter): predict each video directly ? no
training,
252         so no leakage; per-video scoring is already honest.
253     - **learned, pinned model** (``classifier_model`` set): score every video with the
254         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
255         report "how the shipped artifact does here", not for the promotion decision.
256     - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
257         train on the OTHER videos, predict the held-out one. The number to trust.
258     - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
259     """
260     for vc in videos:
261         if vc.gts is None:
262             raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
263
264     per_video: List[Dict[str, Any]] = []
265     predictions: Dict[str, List[Interval]] = {}
266
267     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
268     pinned_model = (LogisticRegression.load(variant.classifier_model)
269                     if variant.classifier_model is not None else None)
270     all_model = None
271     if recipe_lovo and not honest:
272         all_model = _train(variant, videos, iou)
273
274     for i, vc in enumerate(videos):
275         if recipe_lovo and honest:
276             others = [v for j, v in enumerate(videos) if j != i]
277             if not others:
278                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
279             model: Optional[LogisticRegression] = _train(variant, others, iou)
280             elif recipe_lovo: # honest=False ? one model over all
videos
281                 model = all_model
282             else: # static variant or pinned model
283                 model = pinned_model
284             preds = predict(variant, vc, model=model)
285             assert vc.gts is not None # guarded at function top
286             sc = score(preds, vc.gts, iou)
287             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
288             predictions[vc.name] = preds
289
290     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
291                  for m in _METRICS}
292     return {
293         "variant": variant.name,
294         "spec_hash": variant.spec_hash(),
295         "is_learned": variant.is_learned(),
296         "honest_lovo": recipe_lovo and honest,
297         "per_video": per_video,
298         "aggregate": aggregate,
299         "predictions": predictions,
300     }
301
302
303     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,

```



```

304         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
305     """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
306     golden corpus and reports the **B ? A** deltas, per video and in aggregate.
307
308     The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
309     learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
310     `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
311     """
312     if len(videos) < 1:
313         raise ExperimentError("compare needs at least one labeled video")
314     eval_a = evaluate_variant(videos, variant_a, iou, honest)
315     eval_b = evaluate_variant(videos, variant_b, iou, honest)
316
317     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
318     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
319
320     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
321     per_video_delta = [
322         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
323         for p in eval_b["per_video"]
324     ]
325
326     # One fingerprint over the whole label set ? detects "the deltas were measured
against
327     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
328     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
329
330     return {
331         "iou": iou,
332         "label_set_hash": gt_hash(all_gts),
333         "num_videos": len(videos),
334         "variant_a": eval_a,
335         "variant_b": eval_b,
336         "delta": delta,
337         "delta_pct": delta_pct,
338         "per_video_delta": per_video_delta,
339     }
340
341
342     # ----- SxS on unlabeled video
343
344
345     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
346         agree_iou: float = 0.5) -> Dict[str, Any]:
347         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
348
349         With no labels there is no lift to measure ? instead this surfaces where the two
350         variants **agree vs diverge**, which is exactly what a human reviews (and what two
351         highlight reels would show side by side):
352
353         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
354         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
355         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
356             the human / a later golden label adjudicates).
357
358         Both variants must be predictable without labels: a learned variant therefore needs
a
359         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
360         `metrics.match_intervals` ? no new matching logic.

```

```

361     """
362     preds_a = predict(variant_a, video)
363     preds_b = predict(variant_b, video)
364     mr = match_intervals(preds_a, preds_b, agree_iou)
365
366     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
367               for m in mr.matches]
368     agree_ious = [m.iou for m in mr.matches]
369     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
370     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
371
372     def _dur(ivs: List[Interval]) -> float:
373         return sum(e - s for s, e in ivs)
374
375     return {
376         "video": video.name,
377         "agree_iou": agree_iou,
378         "variant_a": variant_a.name,
379         "variant_b": variant_b.name,
380         "num_a": len(preds_a),
381         "num_b": len(preds_b),
382         "num_agreed": len(agreed),
383         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
384         "duration_a": _dur(preds_a),
385         "duration_b": _dur(preds_b),
386         "agreed": agreed,
387         "a_only": a_only,
388         "b_only": b_only,
389     }
390
391 # ----- leaderboard / DB
392
393
394
395 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
396                      timestamp: Optional[str] = None) -> Dict[str, Any]:
397     """Append a compact, reproducible record of a `compare()` result to the JSON
398     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
399     `regression_baseline.json`; deliberately the size of a baseline file, not a service
400     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
401     """
402     row: Dict[str, Any] = {
403         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
404         "iou": result.get("iou"),
405         "label_set_hash": result.get("label_set_hash"),
406         "num_videos": result.get("num_videos"),
407         "variant_a": {"name": result["variant_a"]["variant"],
408                      "spec_hash": result["variant_a"]["spec_hash"],
409                      "aggregate": result["variant_a"]["aggregate"]},
410         "variant_b": {"name": result["variant_b"]["variant"],
411                      "spec_hash": result["variant_b"]["spec_hash"],
412                      "aggregate": result["variant_b"]["aggregate"]},
413         "delta": result.get("delta"),
414     }
415     entries: List[Dict[str, Any]] = []
416     if os.path.isfile(path):
417         with open(path, encoding="utf-8") as f:
418             entries = json.load(f)
419     entries.append(row)
420     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
421     with open(path, "w", encoding="utf-8") as f:
422         json.dump(entries, f, indent=2)
423     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
424               row["variant_a"]["name"], row["variant_b"]["name"],
425               (result.get("delta") or {}).get("f1", 0.0), path)

```

```

426         return row
427
428
429     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
430                             gts: Optional[List[Interval]] = None,
431                             feature_names: Sequence[str] = ()) -> VideoCandidates:
432         """Bridge persisted detection ? `VideoCandidates` via
433         `Database.query_candidate_segments`.
434         ``feature_names`` selects which persisted features to lift out of each row's
435         ``stats``
436         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
437         touching
438         function; the comparison itself is pure and offline. Exercised owner-side on a real
439         DB,
440         but works against any `Database` (including an in-memory one in tests).
441         """
442         rows = db.query_candidate_segments(video_id, detector=detector)
443         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
444         rows]
445         features: Dict[str, List[float]] = {}
446         for name in feature_names:
447             if name == "duration":
448                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
449         rows]
450             else:
451                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
452         rows]
453         return VideoCandidates(name=video_id, intervals=intervals, features=features,
454         gts=gts)
455

```

## 9. assistant (2026-06-13T20:00:23.626Z)

The experiment harness is exactly as the maps describe. Now let me check the manifest, distill\_local, fusion\_features (person feature names), the audio dir contents, and the existing run\_phase3\_eval to find the cleanest seam.

## 10. user (2026-06-13T20:00:24.776Z)

```

1      [
2      {
3          "name": "adarsh_avi_singles",
4          "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
5          "fps": 59.94, "frame_width": 1920,
6          "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh
and Avi.rallies.csv"
7      },
8      {
9          "name": "kushagra_singles",
10         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/kushagra_traj.csv",
11         "fps": 59.94, "frame_width": 1920,
12         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv"
13     },
14     {
15         "name": "largetest_doubles",
16         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/largetestvideo_traj.csv",
17         "fps": 29.97, "frame_width": 1920,
18         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv"
19     },

```

```

20     {
21         "name": "gbaaddy",
22         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/gbaaddy_traj.csv",
23         "fps": 59.94, "frame_width": 1920,
24         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
25     },
26     {
27         "name": "testlarge_short",
28         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
29         "fps": 29.97, "frame_width": 1920,
30         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
31     },
32     {
33         "name": "mahadevpura_singles",
34         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
35         "fps": 29.97, "frame_width": 1920,
36         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
37     }
38 ]
39

```

## 11. user (2026-06-13T20:00:25.184Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score

```

```

36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41     # Recall-oriented proposal: deliberately permissive so real rallies are among the
42     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43     PROPOSAL = (0.005, 1.0, 0.5)
44     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47     def build_candidates(trjectory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50     points = TrackNetRunner.parse_trajectory_csv(trjectory_csv)
51     vt, mg, mwd = proposal
52     wins = TrackNetRunner.trajectory_to_action_windows(
53         points, fps=fps, frame_width=frame_width,
54         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55     intervals = [(w["start"], w["end"]) for w in wins]
56     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
57     X: List[List[float]] = []
58     for w, g in zip(wins, gated):
59         dur = max(1e-6, w["end"] - w["start"])
60         win_frames = max(1.0, dur * fps)
61         X.append([
62             dur, # rally length (sec) ?
63             float(w["peak_velocity"]), # already normalized by
64             float(w["mean_velocity"]),
65             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
66             float(g.crossings), # net crossings (count) ?
67         ])
68     return intervals, X
69
70
71     def baseline_windows(trjectory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
72     points = TrackNetRunner.parse_trajectory_csv(trjectory_csv)
73     vt, mg, mwd = BASELINE_LOCAL
74     wins = TrackNetRunner.trajectory_to_action_windows(
75         points, fps=fps, frame_width=frame_width,
76         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
77     return [(w["start"], w["end"]) for w in wins]
78
79
80     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81         fps, fw = float(spec["fps"]), float(spec["frame_width"])
82         intervals, X = build_candidates(spec["trajectory"], fps, fw)
83         gts = load_golden_gts(spec["labels"])
84         y = label_candidates(intervals, gts, iou)
85         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:

```

```

List[Interval],
90             threshold: float = 0.5) -> List[Interval]:
91     if not intervals:
92         return []
93     proba = model.predict_proba(X)
94     pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95     return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99             model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
123                       f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124             def mean(xs, a):
125                 return sum(getattr(s, a) for s in xs) / len(xs)
126             print(f"  MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127                   f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128                   f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
129             result["mean_learned_f1"] = mean(clf, "f1")
130             result["mean_baseline_f1"] = mean(base, "f1")
131
132             # Final model on ALL videos (for production / running on a fresh held-out video).
133             Xall = [row for v in videos for row in v["X"]]
134             yall = [t for v in videos for t in v["y"]]
135             final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136             assert final.w is not None # set by fit()
137             weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138             print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139             if model_out:
140                 final.save(model_out)
141                 print(f"final model (all videos) -> {model_out}")
142                 result["model_out"] = model_out
143             return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,

```

```

149             help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

## 12. assistant (2026-06-13T20:00:29.200Z)

The manifest is trajectory-CSV-based (not video\_id/DB-based as the first map assumed). This is the critical pivot: `distill\_local` already operates on trajectory CSVs directly, NOT via the DB. The cleanest P3 driver mirrors `distill\_local.load\_video` but adds person features. Let me check fusion\_features, person\_detector signature, and the audio dir.

## 13. user (2026-06-13T20:00:29.944Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of **scale/translation-invariant**
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is ``(x1, y1, x2, y2)`` in **pixels** ; ``(track_id, box)`` when motion
needs it.
12    - ``per_frame`` is ``{frame_idx: [(track_id, box), ...]}`` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's ``.frame`` ? no clip re-
timing).
14    - A shuttle point has ``.frame``, ``.visible``, ``.x``, ``.y`` (pixels) ? the WASB
trajectory shape.
15    - ``court_polygon`` is normalised 0-1 ``(x, y), ...`` or ``None`` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17    - ``net`` is ``(axis 'x'|'y', position 0-1 normalised)`` or ``None`` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except ``mean_player_motion`` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""

```

```

33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71         point meaningless, so a masked check returns False rather than testing a bogus
72         (0,0)."""
73         if court_polygon is None:
74             return True
75         if frame_width <= 0 or frame_height <= 0:
76             return False
77         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                        frame_height: float,
81                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82         """Per-frame count of in-court persons (one entry per sampled frame)."""
83         return [
84             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85             court_polygon))
86             for _frame_idx, dets in sorted(per_frame.items())
87         ]
88
89     def _median(vals: Sequence[float]) -> float:
90         if not vals:
91             return 0.0
92         s = sorted(vals)

```



```

93     n = len(s)
94     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97 def players_in_court(counts: Sequence[int]) -> float:
98     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99     return _median(list(counts))
100
101
102 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103     """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105     if not counts:
106         return 0.0
107     return sum(1 for c in counts if c >= min_players) / len(counts)
108
109
110 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
111                        frame_width: float, frame_height: float) -> float:
112     """Mean per-track centre displacement between consecutive sampled frames, with x
113 normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
114 not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
115     if frame_width <= 0 or len(per_frame) < 2:
116         return 0.0
117     frames = sorted(per_frame.keys())
118     steps: List[float] = []
119     for a, b in zip(frames, frames[1:]):
120         prev = {tid: box for tid, box in per_frame[a]}
121         for tid, box in per_frame[b]:
122             if tid in prev:
123                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
124                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
125                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
126     if not steps:
127         return 0.0
128     return sum(steps) / len(steps)
129
130
131 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
132                     frame_height: float, net: Optional[Net],
133                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
134     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
135
136 A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
137 The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
138 0.0.
139 """
140     if net is None or not per_frame:
141         return 0.0
142     axis, pos = net
143     both = 0
144     for _frame_idx, dets in per_frame.items():
145         near, far = False, False
146         for _tid, box in dets:
147             if not person_in_court(box, frame_width, frame_height, court_polygon):
148                 continue
149             fx, fy = _foot_norm(box, frame_width, frame_height)
150             coord = fx if axis == "x" else fy
151             if coord < pos:
152                 near = True
153             else:
154                 far = True
155         if near and far:
156             both += 1
157     return both / len(per_frame)

```

```

156
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                     pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                               frame_width: float, frame_height: float,
170                               pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174     """
175     if not per_frame or frame_width <= 0 or frame_height <= 0:
176         return 0.0
177     aligned = 0
178     colocated = 0
179     for p in shuttle_points or []:
180         if not getattr(p, "visible", False):
181             continue
182         dets = per_frame.get(int(p.frame))
183         if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184             continue
185         aligned += 1
186         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187               for _tid, box in dets):
188             colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192 # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193 # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194 PERSON_FEATURE_NAMES = [
195     "players_in_court", "two_sided_motion", "mean_player_motion",
196     "in_court_ratio", "shuttle_player_colocation",
197 ]
198
199
200 def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                               frame_width: float, frame_height: float,
202                               court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                               net: Optional[Net] = None,
204                               min_players: int = 2) -> Dict[str, float]:
205     """Aggregate the per-window person features from per-frame detections + the in-
window
206     shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208     return {
209         "players_in_court": players_in_court(counts),
210         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212         "in_court_ratio": in_court_ratio(counts, min_players),
213         "shuttle_player_colocation": shuttle_player_colocation(

```

```

214         per_frame, shuttle_points, frame_width, frame_height),
215     }
216

```

#### 14. assistant (2026-06-13T20:00:32.792Z)

Now the person\_detector signature, the audio dir, and how fusion\_hybrid wires person features. Let me read those.

#### 15. user (2026-06-13T20:00:34.228Z)

```

250         windows.append(_finalize(group))
251
252         # Filter out extremely short action windows
253         min_window_duration = self.config.get("indexing",
254         {}).get("min_action_window_duration", 2.0)
255         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257         return windows
258
259     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
260                             frame_skip: int = 3) -> Dict[str, Any]:
261         """Per-frame person detections over `[start_frame, end_frame)` of the ORIGINAL
262         video.
263         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
264         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
265         discards the
266         per-frame boxes), this returns the raw per-frame detections so the fusion
267         features can
268         align them with the shuttle trajectory. It reads the source by **absolute frame
269         index**
270         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
271         match
272         the WASB trajectory's ``frame`` exactly ? that alignment is what
273         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
274
275         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
276         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
277         seek
278         accuracy varies by codec (it lands near the requested keyframe then decodes
279         forward);
280         treat the indices as ±a few frames on real footage ? fine for window-level
281         features.
282         """
283         self.lazy_load_model()
284         cap = cv2.VideoCapture(video_path)
285         if not cap.isOpened():
286             logger.error(f"Could not open {video_path}")
287             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
288
289         fps = cap.get(cv2.CAP_PROP_FPS)
290         if fps <= 0:
291             fps = 30.0
292         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
293         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
294         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
295         0.5)
296
297         step = max(frame_skip, 1)
298         tracker = self.make_tracker(fps / step)
299
300         start_frame = max(0, int(start_frame))
301         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
302         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
303         frame_idx = start_frame

```

```

294         while frame_idx <= end_frame:
295             ret, frame = cap.read()
296             if not ret:
297                 break
298             if (frame_idx - start_frame) % step == 0:
299                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300                 frame_idx += 1
301         cap.release()
302         return {"frames": frames, "fps": fps,
303                "frame_width": float(frame_width), "frame_height": float(frame_height)}
304

```

## 16. user (2026-06-13T20:00:35.519Z)

```

__init__.py
__pycache__
e1_probe.py
---hit_detector head---
"""Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).

```

Stages 0?1 of docs/AUDIO\_RALLY\_DETECTION\_PLAN.md: extract the audio track, build a spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient a racket?shuttle contact makes), threshold it adaptively, and return (timestamp, strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess) only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.

```

from __future__ import annotations

import os
import wave
import tempfile
import subprocess
from dataclasses import dataclass
from typing import List, Optional, Tuple

import numpy as np

```

```

@dataclass
class HitEvent:
    t: float          # seconds
    strength: float    # onset-envelope peak value (arbitrary units)

```

```

def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) -> str:
    """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
    Audio-only decode, so it's fast even for multi-GB video."""

```

## 17. assistant (2026-06-13T20:00:39.790Z)

Now let me check the exact `detect\_hits\_in\_video` signature and HitEvent, plus how the manifest maps names to video paths (the second map says video files need to be added). Let me verify the video file existence and check fusion\_hybrid's person wiring + the eval/audio/e1\_probe for the existing audio harness pattern.

## 18. user (2026-06-13T20:00:41.940Z)

```

136: def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
    ---
        return []
    # enforce min inter-peak gap: greedily keep strongest first
    min_gap = max(1, int(round(min_gap_sec / hop_sec)))
    order = peak_idx[np.argsort(-env[peak_idx])]

```

```

taken = np.zeros(len(env), dtype=bool)
kept = []
for i in order:
    lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
    if taken[lo:hi].any():
        continue
    taken[i] = True
    kept.append(i)
kept.sort()
return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]

def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
    """Convenience: extract audio from a video and detect hits. Cleans up the temp wav."""
    wav = extract_audio(video_path, sr=sr)
    try:
        samples, real_sr = load_wav(wav)
        return detect_hits(samples, real_sr, **kw)
    finally:
        try:
            os.remove(wav)
        except OSError:
            pass

```

**19. user (2026-06-13T20:00:42.003Z)**

```

1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13     torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14     model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15     via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16     - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
experiment
20     harness can re-score any variant.
21
22     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26     experiment harness (EXPERIMENT_HARNESS.md), never silently.
27     """
28     from typing import Any, Dict, List, Optional, Tuple
29

```

```

30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
48 otherwise).
49             self._person: Optional[Any] = None
50             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
51 estimate it
52             # over the whole trajectory once per candidate window (it depends only on
53 `points`).
54             self._axis_cache_key: Optional[int] = None
55             self._axis_cache: Optional[tuple] = None
56
57         @property
58         def name(self) -> str:
59             return "fusion_hybrid"
60
61         @property
62         def description(self) -> str:
63             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
64 rallies, "
65                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66 and the "
67                    "AI provider sets semantic boundaries.")
68
69         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
70 Any]]:
71             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
72             if substrate == "tracknet":
73                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
74                 return TrackNetRunner(cfg), {
75                     "weights_path": cfg.weights_path,
76                     "queue_length": cfg.queue_length,
77                     "fusion_substrate": "tracknet",
78                 }
79             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
80             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
81 "wasb"}
82
83         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84 frame_width: float, video_path: str,
85 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
87 video_path, idx_cfg)
88             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
89 persisted
90             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
91 estimated
92             # over the whole trajectory by rally_gate (camera-agnostic); reused across
93 windows.

```

```

83         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                           min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                         frame_width: float, video_path: str,
105                         fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                            window_stats: List[Dict[str, Any]],
135                            idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default

```

```

thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are
138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {})).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150                 keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

## 20. assistant (2026-06-13T20:00:47.820Z)

I now have the full picture. The critical insight: the golden manifest is **trajectory-CSV-based** (``distill_local`-style`), NOT DB/video\_id-based. So the cleanest P3 driver mirrors ``distill_local.build_candidates` + `load_video``, computes person features per-window via ``PersonDetector.detections_per_frame`` (reusing existing trajectory CSVs, no WASB re-run), persists a per-video feature JSON, then feeds ``experiment.compare()`` with the two Variants.

Let me confirm: do the trajectory CSVs and video files in the manifest actually exist, and what columns the trajectory CSV has (for ``.frame/.visible/.x/.y`` and absolute frame alignment)?

## 21. user (2026-06-13T20:00:51.906Z)

```

=== Trajectory CSVs ===
EXISTS: adarsh_avi_full_traj.csv
EXISTS: kushagra_traj.csv
EXISTS: largetestvideo_traj.csv
EXISTS: gbaaddy_traj.csv
EXISTS: testlarge_traj.csv
EXISTS: mahadevpura_traj.csv
=== Golden Labelled videos + label CSVs ===
Adarsh and Avi.MP4
Adarsh and Avi.rallies.csv
KushagraYashAviFirstVideo.MP4
KushagraYashAviFirstVideo.rallies.csv
KushagraYashAviFirstVideo.report.json
KushagraYashAviFirstVideo.report.md
KushagraYashAviFirstVideo.summary.md
LargeTestVideo.MP4
LargeTestVideo.rallies.csv
MahadevpuraSingles.MP4
MahadevpuraSingles.rallies.csv
MahadevpuraSingles.report.json
MahadevpuraSingles.report.md
MahadevpuraSingles.summary.md
TestLargeVideo.MP4
TestLargeVideo.rallies.csv
gBaaddy20May2026.MP4
gBaaddy20May2026.rallies.csv

```

## 22. assistant (2026-06-13T20:00:56.870Z)



All 6 trajectory CSVs and 6 videos + label CSVs exist. The manifest only lacks a `video\_path` field per entry (needed for `detections\_per\_frame`). Let me verify the trajectory CSV columns (frame alignment) and check `parse\_trajectory\_csv` + `load\_golden\_gts` shapes, plus confirm there's no existing fusion-LOVO driver already.

### 23. user (2026-06-13T20:01:00.107Z)

```
=== trajectory CSV header + first rows ===
Frame,Visibility,X,Y
1,1,1243.9209365844727,474.6978950500488
2,1,1245.0727844238281,474.86961364746094
=== label CSV header ===
rally_number,start_time,end_time,ending_reason,sport
1,5.654,8.425,service_fault,badminton
2,12.928,18.673,unforced_error,badminton
=== parse_trajectory_csv + Point shape ===
# ----- CSV -> windows

@staticmethod
def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
    """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
    points: List[TrajectoryPoint] = []
    with open(csv_win_path, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                points.append(TrajectoryPoint(
                    frame=int(row["Frame"]),
                    visible=int(row["Visibility"]) == 1,
                    x=float(row["X"]),
                    y=float(row["Y"]),
                ))
            except (KeyError, ValueError):
                continue
    points.sort(key=lambda p: p.frame)
    return points

@staticmethod
def trajectory_to_action_windows(
    points: List[TrajectoryPoint],
    fps: float,
    frame_width: float,
    chunk_start: float = 0.0,
    velocity_thresh: float = 0.02,
    merge_gap: float = 2.0,
    min_window_duration: float = 2.0,
    chunk_index: Optional[int] = None,
```

### 24. user (2026-06-13T20:01:03.652Z)

```
=== load_golden_gts ===
33: def load_golden_gts(csv_path: str) -> List[Interval]:
def load_golden_gts(csv_path: str) -> List[Interval]:
    """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.

    Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
    start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
    driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
    """
    from backend.eval.gt_loader import load_gt_intervals
    return load_gt_intervals(csv_path, strict=False)

def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) -> Callable[[Config],
List[Interval]]:
```

```

"""predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)

def predict_fn(cfg: Config) -> List[Interval]:
    vt, mg, mwd = cfg
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
    )
    return [(w["start"], w["end"]) for w in wins]

return predict_fn

```

```

=== any existing fusion driver / fusion_compare? ===
ls: cannot access 'backend/eval/fusion*': No such file or directory
ls: cannot access 'run_fusion*': No such file or directory
run_phase3_eval.py
=== run_all_tests + test files for experiment ===
test_distill_local.py
test_experiment.py
test_fusion_features.py
test_fusion_hybrid.py
test_person_detector.py

```

## 25. assistant (2026-06-13T20:01:14.017Z)

I now have everything I need. The picture is complete and self-consistent:

- Trajectory CSV columns are `Frame,Visibility,X,Y` ? aligns with `TrajectoryPoint(.frame/.visible/.x/.y)` and `detections\_per\_frame`'s absolute-frame indexing. Frame numbering is 1-based in CSV; `detections\_per\_frame` seeks by absolute frame index. (A real alignment nuance to flag.)
- `load\_golden\_gts` / `gt\_loader.load\_gt\_intervals` reads the label CSVs.
- No fusion driver exists yet. Tests exist for experiment/fusion\_features/distill/person.
- The audio path: `detect\_hits\_in\_video` exists; `audio\_features.py` does NOT exist yet (the map proposes it); `eval/audio/e1\_probe.py` is the existing E1 harness.

The cleanest seam, honoring "prefer a small offline eval driver over the full Gemini pipeline" and "reuse shipped machinery": a `distill\_local`-style offline driver that (a) builds shuttle candidates from the existing trajectory CSV, (b) computes the 5 person features per window via `PersonDetector.detections\_per\_frame` over the original video, (c) writes a per-video features JSON, then a pure-CPU LOVO step builds `VideoCandidates` and calls `experiment.compare()`.

Let me verify one more load-bearing detail ? how `distill\_local`'s window dict exposes `peak\_velocity`/`mean\_velocity`/`active\_frame\_count` keys (used for shuttle features) and the test harness entry, then I'll write the plan.

## 26. user (2026-06-13T20:01:17.249Z)

```

=== window dict keys from _finalize ===
300:         "start": group[0][0] / fps + chunk_start,
301:         "end": group[-1][0] / fps + chunk_start,
302:         "active_frame_count": len(group),
303:         "peak_velocity": max(vels),
304:         "mean_velocity": sum(vels) / len(vels),
319:     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
=== run_all_tests entry ===
=== gt_loader.load_gt_intervals returns ===
44: def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:

```

## 27. assistant (2026-06-13T20:01:21.732Z)

All anchors confirmed. The window dict exposes

`start/end/active\_frame\_count/peak\_velocity/mean\_velocity` exactly as  
`distill\_local.build\_candidates` consumes them. I have a complete, verified picture. Let me  
check how `run\_all\_tests.py` discovers tests so the new driver's tests land correctly.

## 28. user (2026-06-13T20:01:23.021Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.
```

A single, discoverable entry point so you (or a reviewer / CI) can verify the  
whole suite before opening a PR:

```
python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
```

Anything after the script name is passed straight through to pytest.

Notes

```
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
  pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
```

```
from __future__ import annotations
```

```
import importlib.util
import subprocess
import sys
```

```
def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False
```

```
def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
```

## 29. assistant (2026-06-13T20:01:33.265Z)

pytest auto-discovers `tests/test\_\*.py`. I have everything verified. Now I'll synthesize the  
concrete plan and return it via StructuredOutput.

The synthesis distilled to its load-bearing decisions:

1. **\*\*Seam choice\*\***: The golden corpus is **\*\*trajectory-CSV + label-CSV\*\*** based  
(`human\_lovo\_manifest.json`), NOT DB/video\_id based. So the maps' Option 2/3  
(`load\_video\_candidates` from DB) is the **\*wrong\*** bridge ? it would require running the full  
FusionSegmenter+DB pipeline. The right bridge mirrors  
`distill\_local.build\_candidates`/`load\_video`: build shuttle candidates from the CSV, add person  
features via `detections\_per\_frame`, build `VideoCandidates` in-memory, feed  
`experiment.compare()`. This honors "prefer a small offline eval driver over the full Gemini  
pipeline."

2. **\*\*Split\*\***: GPU touches ONLY person-feature extraction (`detections\_per\_frame` over original  
video). That's an offline extractor writing per-video JSON. Everything downstream (LOVO compare)

is pure-CPU and runs in the dev container. So PR1 = extractor + harness driver + tests (tracked, mergeable on green CI per the merging working-agreement); the GPU run produces the JSON artifacts ? number. PR2 = audio cue (`audio\_features.py` + extractor extension + tests).

### 30. user (2026-06-13T20:03:10.775Z)

Output does not match required schema: root: must NOT have additional properties

### 31. assistant (2026-06-13T20:03:13.933Z)

I included an extra `p3\_steps\_note` key. Removing it.

### 32. user (2026-06-13T20:04:27.460Z)

Structured output provided successfully

### 33. assistant (2026-06-13T20:04:37.944Z)

Plan delivered via StructuredOutput. Summary of what I verified and synthesized:

**\*\*Key correction to the maps:\*\*** Maps 1 and 3 assumed the 6 golden videos live in the DB (bridge via `load\_video\_candidates(db, video\_id)`). They do not ? `output/human\_lovo\_manifest.json` is **\*\*trajectory-CSV + label-CSV\*\*** based, and `distill\_local.py` already runs the exact shuttle-only LOVO on it. So the cleanest seam is to mirror `distill\_local`'s CSV flow and add person features, not to stand up the DB path or run the full Gemini pipeline.

**\*\*The split (clean PR vs GPU number):\*\***

- **\*\*PR1 (tracked, mergeable on green CI):\*\*** `backend/eval/fusion\_golden.py` (offline person-feature extractor) + `backend/eval/fusion\_compare.py` (pure-CPU LOVO driver calling shipped `experiment.compare()`) + tests, all GPU-free via a monkeypatchable `detections\_per\_frame` seam. Plus a one-field `video\_path` addition to the manifest.
- **\*\*GPU run (data step, not a PR):\*\*** run `fusion\_golden` on the box ? 6  
`\*\_golden\_features.json`; reuses the existing trajectory CSVs so **\*\*no WASB re-run\*\***. Then `fusion\_compare` in-container prints the honest LOVO ?F1.
- **\*\*PR2:\*\*** `audio\_features.py` + extractor `--audio` branch + tests, structurally identical, default-OFF; audio measured as an *\*incremental\** delta on top of person.

**\*\*No special-casing:\*\*** person and audio enter only as feature columns the `LogisticRegression` weights via `Variant.feature\_names`; promotion is gated on the measured number.

Five open questions flagged, the load-bearing one being the **\*\*1-based (CSV) vs 0-based (cv2 seek) frame-index convention\*\*** that `shuttle\_player\_colocation`/`two\_sided\_motion` alignment depends on ? cheap to confirm on the box, easy to get wrong.